

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ЯДЕРНЫЙ УНИВЕРСИТЕТ
«МИФИ»

ИНСТИТУТ ИНТЕЛЛЕКТУАЛЬНЫХ КИБЕРНЕТИЧЕСКИХ СИСТЕМ

КАФЕДРА «МАШИННОЕ ОБУЧЕНИЕ»

На правах рукописи

УДК _____

Пападык Всеволод Дмитриевич

**«Построение и применение композитных векторных представлений
текста для многоклассовой классификации текстов»**

Выпускная квалификационная работа магистра

Направление подготовки 01.04.02
«Прикладная математика и информатика»

Выпускная
квалификационная работа
защищена
«__»_____ 2026
г.
Оценка

Секретарь ГЭК

г. Москва
2026

РЕФЕРАТ

КОМПОЗИТНЫЕ ВЕКТОРНЫЕ ПРЕДСТАВЛЕНИЯ, МНОГОКЛАССОВАЯ КЛАССИФИКАЦИЯ ТЕКСТОВ, АУГМЕНТАЦИЯ ДАННЫХ, TF-IDF, RUBERT

Объектом исследования являются методы автоматической многоклассовой классификации текстов в условиях ограниченного и несбалансированного корпоративного датасета.

Целью работы является исследование и реализация различных методов построения векторных представлений текста, включая подготовку к использованию композитных эмбедингов, и оценка их влияния на качество многоклассовой классификации с учётом совместной предобработки и аугментации данных.

В процессе работы проведён разведочный анализ исходных данных, выявлены структура и дисбаланс классов, разработаны и реализованы процедуры предобработки и очистки текстов. Реализованы два сценария текстовой аугментации для малочисленных классов: перефразирование и обратный перевод, позволяющие выровнять распределение классов и увеличить вариативность формулировок.

Построены и исследованы модели классификации на основе классического пайплайна TF-IDF + LinearSVC и нейросетевой модели ruBERT, обученные на исходном и аугментированных наборах данных. Выполнено сравнение моделей по набору метрик (accuracy, balanced accuracy, макро- и взвешенная F1-мера), а также проведён анализ ошибок и трудных классов. На основе полученных результатов сформулированы подходы к дальнейшему построению композитных векторных представлений текстов для улучшения качества классификации.

Основные характеристики разработанного прототипа включают модульную архитектуру пайплайна (предобработка, аугментация, построение эмбедингов, обучение и оценка моделей), возможность сравнения классических и нейросетевых подходов и готовность к расширению за счёт внедрения композитных эмбедингов.

Степень внедрения результатов работы определяется возможностью адаптации разработанного прототипа к задачам автоматической классификации корпоративных текстов, в частности маршрутизации обращений и категоризации внутренних документов. Полученные наработки и программная реализация могут быть использованы в качестве основы для последующей интеграции в промышленные системы анализа текстовых данных.

Ключевые слова: МНОГОКЛАССОВАЯ КЛАССИФИКАЦИЯ, ТЕКСТОВЫЕ ЭМБЕДДИНГИ, КОМПОЗИТНЫЕ ПРЕДСТАВЛЕНИЯ, АУГМЕНТАЦИЯ ТЕКСТОВ, TF-IDF, RUBERT, НЕЙРОСЕТЕВЫЕ МОДЕЛИ.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	6
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	8
ВВЕДЕНИЕ	10
1 Обзор предметной области	12
1.1 Многоклассовая классификация текстов и её приложения.....	12
1.2 Особенности классификации длинных русскоязычных деловых писем	12
1.3 Векторные представления текста: от Bag-of-Words до контекстных эмбеддингов	13
1.4 Проблемы разреженности, дисбаланса классов и качества текстовых представлений	13
1.5 Методы аугментации текстовых данных	14
1.6 Подходы к обработке длинных текстов в transformer-моделях	15
1.7 Идея композитных векторных представлений текста	16
2 Разведочный анализ и предобработка данных	17
2.1 Структура датасета и распределение классов.....	17
2.2 Анализ длины текстов и качества исходных данных.....	17
2.3 Обнаружение дубликатов, пересечений и аномальных объектов.....	18
2.4 Правила очистки и нормализации текстов.....	19
2.5 Итоговое состояние данных после предобработки	20
2.6 Особенности train/test-разбиения и распределения классов.....	21
3 Постановка задачи и проектирование решения.....	23
3.1 Формальная постановка задачи многоклассовой классификации	23
3.2 Требования к разрабатываемому решению.....	23
3.3 Влияние предметной области на проектирование решения.....	23
3.4 Общая архитектура пайплайна обработки и классификации текста	23
3.5 Выбор направлений моделирования	24
3.6 Использование суммаризации для подготовки дополнительных представлений текста.....	25
3.7 Построение суммаризованного и комбинированного наборов данных	25
3.8 Выбор метрик качества классификации	26
4 Аугментация данных и обогащение выборки.....	27
4.1 Мотивация аугментации при дисбалансе классов.....	27
4.2 Аугментация классическими методами без использования нейросетевых моделей	27
4.3 Общая схема нейросетевой аугментации длинных текстов	27
4.4 Разбиение текста на предложения и фрагменты при аугментации	27
4.5 Генерация нескольких вариантов аугментации и выбор итогового кандидата.....	28
4.6 Схема доведения редких классов до порогового размера выборки.....	28
4.7 Аугментация с использованием перефразирования.....	29

4.8	Аугментация методом обратного перевода	29
4.9	Фильтрация аугментированных текстов по косинусному сходству.....	30
4.10	Склейка аугментированных фрагментов в полный текст документа.....	30
4.11	Формирование итогового аугментированного набора данных	30
4.12	Анализ распределения данных после аугментации	31
5	Классические модели на основе разреженных текстовых признаков.....	33
5.1	Теоретические основы TF-IDF-представления текста	33
5.2	Линейные классификаторы для разреженных текстовых признаков	33
5.3	Построение признаков на основе word n-грамм	34
5.4	Построение признаков на основе символьных char n-грамм	34
5.5	Комбинирование word- и char-признаков в едином признаковом пространстве	34
5.6	Реализация пайплайна TF-IDF + LinearSVC	35
5.7	Реализация пайплайна TF-IDF + LogisticRegression	35
5.8	Дополнительные классические модели классификации.....	35
5.9	Особенности применения TF-IDF к длинным деловым письмам.....	36
6	Контекстные представления и классификация на основе BERT-моделей	37
6.1	Теоретические основы transformer-архитектур.....	37
6.2	Контекстные эмбединги и их отличие от разреженных признаков.....	37
6.3	Используемые русскоязычные модели.....	38
6.4	Два подхода к использованию BERT в задаче классификации	38
6.4.1	Дообучение Sentence-BERT для построения эмбедингов	39
6.4.2	Дообучение BERT как end-to-end классификатора	39
6.5	Классификация через специальный токен CLS	40
6.6	Классификация через усреднение скрытых состояний токенов (mean pooling).....	40
6.7	Эксперименты с ruBERT-base-cased и различными классификационными головами	41
6.8	Эксперименты с ruRoberta-large и различными классификационными головами	41
6.9	Ограничение длины входной последовательности и проблема длинных документов	41
6.10	Chunked-классификатор: архитектура для длинных документов	42
6.11	Классификационная голова и функция потерь	43
6.12	Параметры обучения chunked-классификатора	43
6.13	Регуляризация и стабилизация обучения на малой выборке	44
7	Композитные и эмбединговые представления текста	46
7.1	Варианты BERT-эмбедингов: предобученные и дообученные.....	46
7.2	Классические классификаторы поверх плотных эмбедингов	46
7.3	Классификация по косинусному сходству: центроиды и ближайшие соседи.....	47
7.4	Конкатенация TF-IDF и BERT-векторов	47
7.5	Реализация композитного признакового пространства	48

7.6 Классические модели и MLP поверх композитных векторов	48
7.7 Композитная ветка как объединение классических и нейросетевых подходов	49
8 Программная реализация разработанного пайплайна	50
8.1 Общая структура программного решения.....	50
8.2 Модуль предобработки данных.....	50
8.3 Модуль аугментации текстов	51
8.4 Модуль построения векторных представлений	51
8.5 Модуль обучения моделей.....	51
8.6 Воспроизводимость экспериментов.....	52
8.7 Ограничения реализации и требования к вычислительным ресурсам	52
9 Экспериментальное исследование и сравнение результатов	54
9.1 Экспериментальный протокол оценки качества.....	54
9.2 Используемые метрики качества классификации	54
9.3 Сравнение качества на исходной и аугментированной обучающей выборке	54
9.4 Сравнение исходных, суммаризованных и комбинированных текстовых представлений	55
9.5 Сравнение классических, нейросетевых и композитных представлений	57
9.6 Сводная таблица результатов лучших реализованных подходов	58
9.7 Выводы по результатам экспериментального исследования	58
ЗАКЛЮЧЕНИЕ.....	60
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	62

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчете применяют следующие термины с соответствующими определениями:

Многоклассовая классификация текста	—	задача автоматического отнесения текстового документа к одному из заранее заданных классов из конечного множества категорий
Векторное представление текста (эмбединг текста)	—	численное представление текста в виде вектора признаков, используемое моделью машинного обучения для классификации или сравнения документов.
Композитное векторное представление текста	—	признаковое представление документа, полученное путём объединения нескольких типов признаков, например TF-IDF-вектора и BERT-эмбединга
Текстовая аугментация	—	совокупность методов увеличения обучающей выборки за счёт генерации дополнительных текстов на основе исходных документов с сохранением их целевой метки
Перефразирование текста	—	метод текстовой аугментации, при котором исходный текст преобразуется в близкий по смыслу вариант с изменённой лексической или синтаксической формой
Обратный перевод	—	метод текстовой аугментации, при котором текст переводится на промежуточный язык, а затем обратно на исходный язык для получения новой формулировки
Базовая модель классификации	—	модель, реализующая простой или широко используемый подход и применяемая как отправная точка для сравнения с более сложными решениями
Нейросетевая модель классификации текста	—	модель классификации, основанная на нейронных сетях и использующая плотные или контекстные представления текста
Корпоративный датасет текстов	—	набор текстовых данных, сформированный на основе документов или переписки организации и отражающий реальные процессы предметной области
Деловое письмо	—	текстовый документ корпоративной переписки, содержащий служебную,

	управленческую, проектную или организационную информацию
Разреженное представление текста	— векторное представление, в котором большинство значений равно нулю; в работе к таким представлениям относятся TF-IDF-векторы на основе word- и char n-грамм
Контекстное эмбединговое представление	— плотное векторное представление текста, полученное с помощью языковой модели и учитывающее контекст употребления слов
Суммаризация текста	— метод построения сокращённого представления документа, сохраняющего его основное содержание
Чанк	— фрагмент длинного текста, полученный при разбиении документа на части ограниченной длины для обработки BERT-подобной моделью
Классификация по центроидам	— метод классификации в эмбединговом пространстве, при котором для каждого класса вычисляется средний вектор, а новый объект относится к классу с наиболее близким центроидом
Классификация методом ближайшего соседа	— метод классификации, при котором метка нового документа определяется по наиболее близкому обучающему примеру в векторном пространстве

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчете применяют следующие сокращения и обозначения:

BERT	—	Bidirectional Encoder Representations from Transformers, языковая модель на основе encoder-части transformer-архитектуры
ruBERT	—	русскоязычная BERT-модель, предварительно обученная на русскоязычных текстах
RoBERTa	—	модификация BERT, использующая изменённую стратегию предварительного обучения
ruRoberta-large	—	русскоязычная RoBERTa-модель большого размера, используемая в работе для классификации и построения эмбедингов
SBERT	—	Sentence-BERT, подход к построению эмбедингов предложений и документов на основе BERT-подобных моделей
TF-IDF	—	Term Frequency - Inverse Document Frequency, метод взвешенного представления текста, учитывающий частоту термина в документе и его распространённость в корпусе
SVM	—	Support Vector Machine, метод опорных векторов
LinearSVC	—	линейный классификатор на основе метода опорных векторов, используемый для работы с разреженными признаками
LogisticRegression	—	логистическая регрессия, линейная модель классификации
MLP	—	Multi-Layer Perceptron, многослойный перцептрон
NLP	—	Natural Language Processing, обработка естественного языка
F1	—	гармоническое среднее между точностью (precision) и полнотой (recall), используемое для оценки качества классификации
Macro F1	—	макро-усреднённая F1-мера, при которой F1 рассчитывается отдельно для каждого класса, после чего значения усредняются с равным весом

Accuracy	—	доля правильно классифицированных объектов в общей выборке
Balanced accuracy	—	сбалансированная точность, рассчитываемая как среднее значение полноты по классам
GPU	—	Graphics Processing Unit, графический процессор, используемый для ускорения вычислений при обучении нейросетевых моделей
EDA	—	Exploratory Data Analysis, первичный исследовательский анализ данных

ВВЕДЕНИЕ

В условиях роста объемов корпоративной текстовой информации всё более востребованными становятся методы автоматической классификации документов и сообщений. Такие методы применяются для маршрутизации обращений, распределения писем по ответственным подразделениям, категоризации внутренних документов и поддержки процессов электронного документооборота. На практике подобные задачи часто имеют многоклассовый характер: документ необходимо отнести не к одной из двух категорий, а к одному из множества возможных направлений, проектов или управленческих блоков.

Особую сложность представляет классификация длинных деловых писем. В отличие от коротких сообщений, такие документы могут содержать несколько смысловых частей, служебные блоки, подписи, пересылаемые фрагменты, формальные обороты и предметно-специфичную лексику. При этом важная для классификации информация может находиться не только в начале текста, но и в середине или конце документа. Это ограничивает применимость простого усечения текста и требует специальных подходов к предобработке, аугментации и построению признаков представлений.

В рассматриваемой задаче используется набор русскоязычных деловых писем, размеченных по 36 классам. Исходные данные характеризуются выраженным дисбалансом: часть классов содержит большое число документов, тогда как отдельные категории представлены малым количеством примеров. Такая структура типична для корпоративной переписки, где одни направления деятельности создают постоянный поток документов, а другие встречаются значительно реже. В этих условиях качество классификации зависит не только от выбранной модели, но и от подготовки данных, выбора метрик, а также способов расширения обучающей выборки.

Традиционные подходы к представлению текста, основанные на TF-IDF, остаются важной базовой линией для задач классификации. Они позволяют эффективно использовать линейные классификаторы, быстро обучаются и хорошо фиксируют характерные слова, n-граммы, аббревиатуры и устойчивые деловые формулировки. Одновременно такие подходы ограниченно учитывают контекст и смысловую близость разных формулировок. Поэтому в работе дополнительно рассматриваются BERT-подобные модели, способные строить контекстные представления текста.

В рамках исследования реализуются несколько направлений моделирования. Классическое направление основано на TF-IDF-признаках и линейных классификаторах. Нейросетевое направление включает ruBERT и ruRoberta-large, в том числе варианты классификации через стандартную голову, mean pooling и отдельную архитектуру для длинных документов с разбиением на чанки. Также рассматриваются эмбединговые подходы, Sentence-BERT-представления, классификация по косинусному сходству, суммаризация документов и композитные признаки, объединяющие TF-IDF и BERT-векторы.

Актуальность работы определяется необходимостью построения воспроизводимого пайплайна для классификации длинных русскоязычных деловых писем в условиях ограниченной и несбалансированной обучающей выборки. Такая задача требует не только выбора модели, но и комплексной организации обработки данных: очистки, анализа распределения классов, аугментации, суммаризации, построения разных признаков пространств и корректного сравнения результатов.

Цель работы заключается в разработке и исследовании пайплайна многоклассовой классификации длинных русскоязычных деловых писем на основе классических, нейросетевых, эмбединговых и композитных представлений текста.

Для достижения поставленной цели в работе решаются следующие задачи:

- 1) провести разведочный анализ исходного датасета, изучить распределение классов, длину документов и качество текстов;
- 2) разработать и реализовать процедуры очистки и нормализации текстов с учётом специфики деловой переписки;
- 3) сформировать фиксированное train/test-разбиение и обеспечить отсутствие использования тестовых данных при обучении и аугментации; построить базовый классический пайплайн классификации на основе TF-IDF и линейного классификатора и оценить его качество;
- 4) реализовать методы обогащения обучающей выборки, включая классическую обработку, перефразирование и обратный перевод;
- 5) построить дополнительные варианты текстовых представлений на основе суммаризации и комбинирования исходного текста с summary;
- 6) реализовать классические модели классификации на основе TF-IDF-признаков, включая word-, char- и объединённые word+char-представления;
- 7) реализовать и исследовать BERT-подходы на основе ruBERT и ruRoberta-large, включая варианты с различными классификационными головами и чанкингом длинных документов;
- 8) построить BERT- и Sentence-BERT-эмбединги документов и исследовать классификацию на их основе;
- 9) реализовать композитное признаковое пространство, объединяющее TF-IDF и BERT-векторы;
- 10) провести экспериментальное сравнение реализованных подходов по метрикам balanced accuracy и macro F1.

Объектом исследования является процесс автоматической многоклассовой классификации русскоязычных деловых писем.

Предметом исследования являются методы предобработки, аугментации, суммаризации и построения векторных представлений текста, применяемые для классификации длинных документов в условиях дисбаланса классов.

Теоретическая значимость работы состоит в систематизации и сравнении нескольких подходов к представлению длинных текстов: разреженных TF-IDF-признаков, контекстных BERT-представлений, Sentence-BERT-эмбедингов и композитных векторов. В работе рассматривается влияние не только модели, но и всего пайплайна обработки данных на итоговое качество классификации.

Практическая значимость работы заключается в разработке модульного программного прототипа, включающего предобработку данных, аугментацию, суммаризацию, построение признаков, обучение моделей и сравнение результатов. Предложенный пайплайн может быть адаптирован для задач автоматической маршрутизации корпоративных писем, категоризации внутренних документов и поддержки систем электронного документооборота.

1 Обзор предметной области

1.1 Многоклассовая классификация текстов и её приложения

Многоклассовая классификация текстов — это задача автоматического отнесения документа к одной из нескольких заранее заданных категорий. Такая задача возникает в тех случаях, когда текстовый поток необходимо распределить по конкретным темам.

На практике многоклассовая классификация используется в системах электронного документооборота, службах поддержки, корпоративных почтовых системах, новостных агрегаторах, юридических и финансовых информационных системах. Например, входящее письмо может быть автоматически отнесено к определённому проекту, отделу, типу запроса или зоне ответственности сотрудника. Это позволяет сократить ручную обработку документов.

В рамках данной работы рассматривается прикладной вариант этой задачи: классификация длинных русскоязычных деловых писем по 36 классам. Документы имеют значительную длину, содержат формальные обороты, подписи, пересылаемые фрагменты, служебные конструкции и предметно-специфичную лексику. Кроме того, отдельные классы могут быть близки по смыслу, так как относятся к смежным подразделениям, проектам или направлениям деятельности. Поэтому модель должна учитывать не только отдельные ключевые слова, но и общий контекст письма.

Дополнительную сложность создаёт ограниченный и несбалансированный набор данных. В исходной обучающей выборке разные классы представлены неравномерно: одни категории содержат большое количество писем, а другие имеют только несколько примеров. В такой ситуации модель может смещаться в сторону крупных классов и хуже распознавать редкие категории. Поэтому в работе задача классификации рассматривается совместно с анализом распределения классов, предобработкой текстов, аугментацией данных и сравнением разных способов построения текстовых представлений.

1.2 Особенности классификации длинных русскоязычных деловых писем

В деловой переписке текст часто строится по более сложной структуре: в одном письме могут присутствовать приветствие, описание ситуации, ссылки на предыдущие договорённости, пересылаемые фрагменты, подписи, реквизиты, уточнения по срокам и исполнителям. При этом не все части письма одинаково полезны для определения класса. Например, подписи и служебные блоки могут повторяться в разных категориях, а ключевая информация может находиться только в одном или нескольких абзацах.

В рассматриваемой задаче документы являются длинными русскоязычными деловыми письмами. Это накладывает несколько ограничений на выбор методов обработки текста. Во-первых, длинные документы сложнее подавать на вход transformer-моделям, так как большинство BERT-подобных архитектур имеет ограничение на длину входной последовательности. Если просто обрезать письмо до фиксированного числа токенов, часть значимой информации может быть потеряна. Во-вторых, длинный текст часто содержит информационный шум: повторяющиеся служебные конструкции, фрагменты переписки, стандартные формулировки и элементы, которые не помогают отличать один класс от другого.

Русский язык дополнительно усложняет задачу за счёт развитой морфологии и свободного порядка слов. Одни и те же сущности, действия и признаки могут встречаться в разных грамматических формах. Для классических моделей это означает увеличение числа возможных токенов и разреженность признакового пространства. Для нейросетевых моделей проблема проявляется иначе: модель может лучше учитывать контекст, но при

малом количестве размеченных примеров не всегда получает достаточно данных для устойчивого дообучения под конкретную предметную область.

Специфика деловой переписки также связана с высокой тематической близостью классов. В отличие от задач, где категории хорошо различимы по лексике, в корпоративных письмах разные классы могут использовать сходные термины, шаблоны и управленческие формулировки. Например, письма, относящиеся к разным подразделениям или проектам, могут содержать одинаковые слова о согласовании, сроках, документах, исполнителях и отчётности. Поэтому классификатору необходимо выделять не только общеделовую лексику, но и более тонкие признаки, связанные с конкретным направлением или контекстом письма.

1.3 Векторные представления текста: от Bag-of-Words до контекстных эмбеддингов

Для применения алгоритмов машинного обучения текст должен быть преобразован в численное представление — вектор признаков фиксированной длины. Самые ранние подходы опирались на модели one-hot и Bag-of-Words, в которых каждый текст описывается вектором, отражающим частоты или наличие слов из словаря. Расширение этой идеи позволяет взвешивать слова с учётом их частоты в документе и редкости в коллекции, снижая влияние часто встречающихся, но малоинформативных терминов и наоборот повышая редкие, но высокоинформативные термины — TF-IDF.

Но такие представления не учитывают порядок слов и семантическую близость, а также страдают от разреженности при больших словарях. Для преодоления этих ограничений были предложены плотные распределённые представления. Такими методами стали Word2Vec, GloVe и FastText, в которых каждая лексема отображается в вектор невысокой размерности, обученный на больших корпусах. Революционным шагом стало появление контекстных эмбеддингов, в том числе на основе архитектуры BERT, которые формируют представление токена или целого текста с учётом контекста. Это позволяет различать многозначные слова и тонкие смысловые нюансы.

Поскольку разные типы представлений фиксируют разные свойства текста, в работе также рассматривается идея композитного признакового пространства. В таком подходе разреженные TF-IDF-признаки объединяются с плотными BERT-векторами, после чего поверх общего представления обучается классификатор. Цель такого подхода заключается в том, чтобы совместить сильные стороны классических и нейросетевых представлений: чувствительность TF-IDF к характерным словам и фразам, а также способность BERT-моделей учитывать контекст и семантическую близость текстов.

Из-за перечисленных особенностей в работе рассматривается несколько способов представления и обработки текстов. Для классического подхода используются TF-IDF-признаки, включая word- и char- ngram-представления, так как они позволяют учитывать как отдельные слова, так и устойчивые фрагменты словоформ. Для нейросетевых моделей исследуются BERT-модели, которые способны строить контекстные представления текста. Отдельно рассматриваются композитные представления, объединяющие разные типы признаков.

1.4 Проблемы разреженности, дисбаланса классов и качества текстовых представлений

При классификации текстов качество модели во многом зависит от того, насколько удачно исходные документы преобразованы в признаковое пространство. В классических подходах, таких как Bag-of-Words и TF-IDF, каждый документ описывается большим количеством признаков, соответствующих словам, n-граммам или символьным фрагментам. Такое пространство обычно получается высокоразмерным и разреженным:

большая часть признаков для конкретного документа имеет нулевые значения. Это естественно для текстовых данных, так как даже длинное письмо содержит только малую часть всех возможных слов и сочетаний, встречающихся в корпусе.

Разреженность сама по себе не делает метод непригодным. Линейные модели часто хорошо работают с такими признаками, поскольку могут выделять информативные слова и устойчивые выражения для каждого класса. Однако при малом количестве обучающих примеров возникают ограничения. Если редкий класс представлен несколькими письмами, модель видит слишком мало вариантов формулировок и может переобучиться на случайные слова, подписи, имена проектов или служебные фрагменты. В результате признаки, которые не отражают сущность класса, могут получить слишком большой вес.

В рассматриваемой задаче эта проблема усиливается дисбалансом классов. Исходная обучающая выборка содержит 36 классов, но количество документов по ним распределено неравномерно: для части классов доступно большое число примеров, а отдельные категории представлены единичными письмами. В такой ситуации модель может показывать приемлемое общее качество за счёт крупных классов, но плохо распознавать малые классы. Поэтому при анализе результатов важно использовать не только общую долю правильных ответов, но и метрики, чувствительные к качеству на каждом классе, например *balanced accuracy* и *macro F1*.

Дисбаланс влияет не только на классические модели, но и на нейросетевые подходы. BERT-подобные модели обладают более выразительными представлениями, однако для дообучения под конкретную предметную область им также требуется достаточное количество размеченных примеров. Если данных мало, а классы близки по тематике, модель может не получить устойчивого сигнала для разделения категорий. Особенно это заметно в задачах с длинными документами, где значимая информация может занимать небольшую часть текста, а остальное содержимое создаёт шум.

Качество текстового представления также зависит от того, какие уровни информации оно учитывает. Словные признаки хорошо отражают явные термины и фразы, но чувствительны к вариативности формулировок. Символьные *n*-граммы помогают работать с морфологией, сокращениями и частичными совпадениями слов. Контекстные эмбединги лучше учитывают смысловые связи, но могут терять часть информации при ограничении длины входа или при недостаточном объёме обучающих данных. Поэтому в данной работе рассматриваются несколько типов представлений: разреженные TF-IDF-признаки, плотные BERT-эмбединги, а также композитные векторы, объединяющие разные источники информации.

Для уменьшения влияния дисбаланса и повышения разнообразия обучающих примеров в работе используется аугментация данных. Она применяется только к обучающей выборке и направлена на расширение представления редких классов за счёт перефразирования и обратного перевода. Дополнительно рассматриваются суммаризованные и комбинированные варианты данных, позволяющие проверить, может ли сжатое представление документа сохранить признаки, важные для классификации. Таким образом, проблемы разреженности, дисбаланса и качества признаков определяют не только выбор моделей, но и общую структуру экспериментального пайплайна.

1.5 Методы аугментации текстовых данных

Аугментация текстовых данных применяется в тех случаях, когда исходная обучающая выборка недостаточна по объёму или имеет выраженный дисбаланс между классами. Основная идея заключается в создании дополнительных обучающих примеров на основе уже размеченных текстов. При этом новые тексты должны сохранять исходную метку класса, но отличаться по формулировкам, структуре или отдельным языковым признакам. Для задачи классификации это позволяет увеличить разнообразие обучающих примеров и снизить зависимость модели от случайных фрагментов исходных документов.

В текстовых задачах аугментация сложнее, чем в изображениях или табличных данных. Небольшое изменение слова может изменить смысл документа, нарушить стиль или сделать текст неестественным. Поэтому для классификации деловых писем важно не просто создать дополнительные варианты текста, а сохранить его основное содержание и принадлежность к тому же классу. Особенно это критично для редких классов, где каждый исходный пример оказывает заметное влияние на обучение модели.

Существуют простые методы аугментации без использования нейросетевых моделей. К ним относятся замена слов синонимами, случайное удаление или перестановка слов, вставка близких по смыслу слов, небольшие изменения пунктуации. Такие методы легко реализуются и не требуют больших вычислительных ресурсов, однако для длинных деловых писем они имеют ограниченную применимость. В подобных документах важны формулировки, терминология и деловой стиль, поэтому случайные изменения могут создавать шум и ухудшать качество обучающих данных.

Более содержательными являются методы нейросетевой аугментации. В данной работе рассматриваются два таких подхода: перефразирование и обратный перевод. Перефразирование направлено на получение нового варианта текста с сохранением исходного смысла. Обратный перевод строится иначе: текст переводится на промежуточный язык, а затем обратно на русский. В результате часть формулировок изменяется, но общий смысл документа должен сохраняться. Оба метода позволяют получить более естественные варианты текста по сравнению с простыми случайными заменами.

Для длинных писем аугментация полного текста целиком может быть затруднена из-за ограничений моделей и высокой длины входной последовательности. Поэтому в работе используется обработка на уровне фрагментов: документ разбивается на предложения и более короткие части, для которых генерируются варианты аугментации. После этого фрагменты объединяются обратно в полный текст. Такой подход позволяет применять модели аугментации к длинным документам и при этом сохранять общую структуру письма.

Важным этапом является контроль качества сгенерированных текстов. Если аугментированный пример слишком близок к исходному, он почти не добавляет новой информации для обучения. Если же он слишком сильно отличается, возникает риск изменить смысл и получить пример с некорректной меткой. Поэтому в работе используется фильтрация по косинусному сходству между исходным и аугментированным текстом. В итоговую обучающую выборку включаются только те варианты, которые сохраняют достаточную смысловую близость к оригиналу, но не являются его почти точной копией.

1.6 Подходы к обработке длинных текстов в transformer-моделях

При работе с длинными документами Transformer-модели имеют важное ограничение: стандартные BERT-подобные архитектуры принимают на вход последовательность ограниченной длины. На практике это означает, что длинное письмо не всегда может быть обработано целиком. Если документ просто обрезается до максимально допустимой длины, часть информации может быть потеряна, причём заранее неизвестно, где именно находится фрагмент, важный для классификации.

Один из практических способов работы с длинными текстами заключается в разбиении документа на несколько фрагментов. Каждый фрагмент подаётся в transformer-модель отдельно, после чего полученные представления объединяются на уровне документа. Фрагменты могут формироваться с перекрытием, чтобы информация на границах частей не терялась. Такой подход позволяет использовать стандартные BERT-подобные модели без изменения их архитектуры и при этом учитывать большую часть исходного письма.

В данной работе этот принцип используется при реализации классификатора на основе `ruRoberta-large` для длинных документов. Текст разбивается на чанки фиксированной длины с заданным шагом, после чего модель обрабатывает несколько фрагментов одного письма. Далее признаки фрагментов агрегируются и передаются в классификационную голову. Такой подход выбран как компромисс между полнотой использования текста и вычислительными ограничениями, поскольку обработка всех возможных фрагментов длинного письма может быть слишком затратной.

Помимо чанкинга, для длинных текстов может применяться суммаризация. В этом случае из исходного документа формируется сокращённая версия, которая должна сохранять основное содержание. Такой вариант полезен для проверки гипотезы о том, достаточно ли краткого представления письма для определения класса. В работе поэтому рассматриваются не только исходные тексты, но и суммаризованные документы, а также комбинированный вариант, где суммаризованный текст используется вместе с оригиналом. Это позволяет сравнить, какую роль играет полный текст и насколько информативным оказывается его сжатое представление.

1.7 Идея композитных векторных представлений текста

Разные способы представления текста фиксируют разные свойства документа. Разреженные признаки, построенные с помощью TF-IDF, хорошо отражают наличие характерных слов, устойчивых словосочетаний и символьных фрагментов. Они особенно полезны в тех случаях, когда класс можно частично определить по специфичной терминологии, названиям проектов, подразделений, типовым формулировкам или повторяющимся деловым оборотам. При этом такие признаки слабо учитывают общий смысл текста и не всегда устойчивы к изменению формулировок.

Контекстные представления, получаемые с помощью BERT-подобных моделей, решают другую задачу. Они позволяют учитывать окружение слов и строить более плотное семантическое описание текста. Благодаря этому близкие по смыслу формулировки могут получать похожие векторы даже при различии в конкретных словах. Для деловой переписки это важно, поскольку один и тот же тип обращения может быть выражен разными фразами. Однако плотные эмбединги не всегда явно сохраняют информацию о редких, но важных токенах, которые могут быть значимыми для конкретного класса.

Идея композитного векторного представления заключается в объединении нескольких источников признаков в одном пространстве. В рамках данной работы это означает конкатенацию разреженного TF-IDF-представления и плотного BERT-вектора. Такое объединение позволяет одновременно использовать лексические признаки и контекстную информацию. TF-IDF-компонента отвечает за точные совпадения слов, n-грамм и символьных шаблонов, а BERT-компонента добавляет информацию о семантике и контексте документа.

После построения композитного вектора задача классификации сводится к обучению модели поверх объединённого признакового пространства. В работе для этого рассматриваются классические классификаторы: `LinearSVC` и `LogisticRegression`, а также MLP-классификатор. Такой подход позволяет проверить, может ли объединение разреженных и плотных признаков дать более информативное описание текста по сравнению с использованием каждого типа представления отдельно.

2 Разведочный анализ и предобработка данных

2.1 Структура датасета и распределение классов

В работе используется набор русскоязычных деловых писем, размеченных по тематическим классам. Каждый объект датасета представляет собой отдельный текстовый документ, которому сопоставлена одна целевая метка. Конфиденциальная информация в документах заменена масками-заполнителями. Метка соответствует направлению, проекту, подразделению или иной категории, используемой для классификации письма. Таким образом, задача рассматривается как классификация одного документа в один из заранее заданных классов.

Всего в наборе данных представлено 36 классов. Такое количество категорий делает задачу более сложной по сравнению с бинарной классификацией или классификацией на небольшое число тем. При этом классы не являются равномерными по объёму: часть категорий содержит большое число документов, а некоторые представлены существенно меньшим количеством примеров, распределение представлено на рисунке 2.1. Для реальных деловых данных такая ситуация является естественной, так как интенсивность переписки по разным направлениям может значительно отличаться.

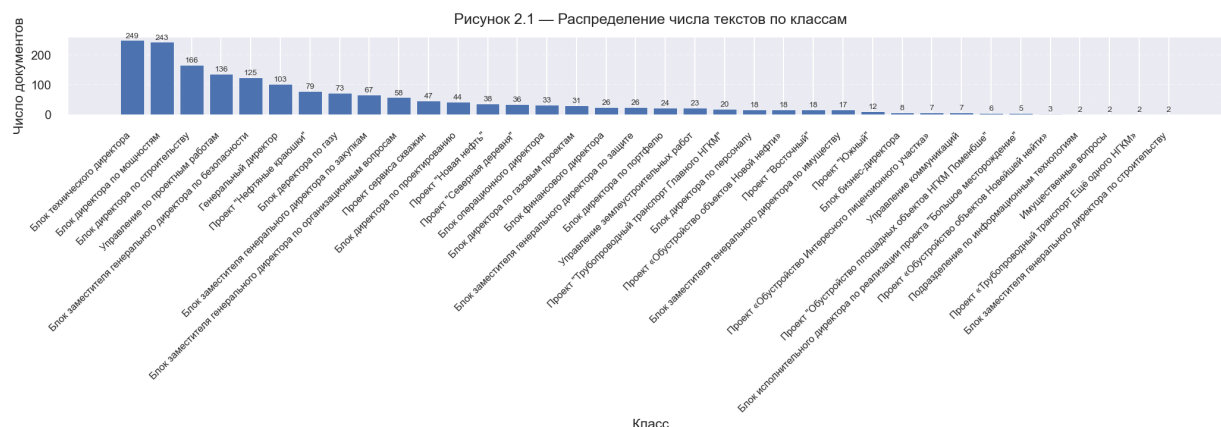


Рисунок 2.1 — Распределение числа текстов по классам

Анализ распределения классов показывает, что датасет имеет выраженный дисбаланс. Наиболее крупные классы содержат существенно больше документов, чем редкие категории. Это означает, что разные классы представлены в обучающих данных с разной степенью полноты: для крупных категорий модель может наблюдать больше вариантов формулировок, а для малых — только ограниченное число примеров. Данная особенность учитывается при выборе метрик качества и при проектировании этапа аугментации.

2.2 Анализ длины текстов и качества исходных данных

В рамках анализа рассматривались базовые характеристики длины документов: количество символов, количество слов и приблизительная длина в токенах. Распределения представлены на рисунках 2.2 и 2.3. Для классических моделей на основе TF-IDF большая длина текста обычно не является техническим ограничением, однако она увеличивает количество потенциальных признаков и может усиливать влияние повторяющихся служебных фрагментов. Для BERT-моделей длина текста является более критичным фактором, так как входная последовательность ограничена фиксированным числом токенов.

Рисунок 2.2 — Распределение длины документов по числу символов

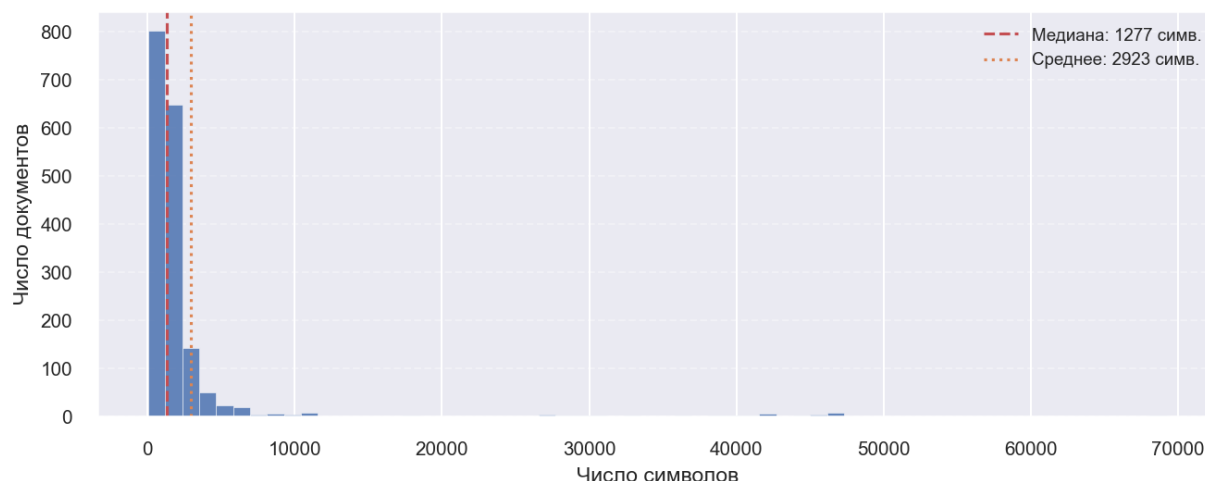


Рисунок 2.2 — Распределение длины документов по числу символов

Рисунок 2.3 — Распределение длины документов (до очистки)

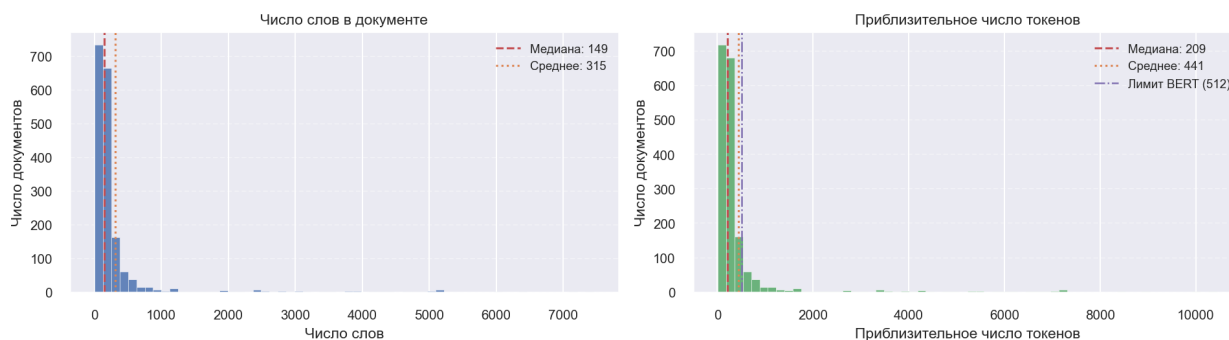


Рисунок 2.3 — Распределение длины документов по числу слов или токенов

Анализ показал, что в наборе данных присутствуют как относительно короткие письма, так и длинные документы. Помимо длины, анализировалось качество исходных данных. В деловой переписке часто встречаются технические элементы, которые не всегда несут полезную информацию для классификации: подписи, приветствия, повторы, служебные символы, ссылки, шаблонные фразы, фрагменты пересланной переписки. Такие элементы могут встречаться в разных классах и создавать шум в признаковом пространстве. При этом полностью удалять все служебные конструкции нельзя, поскольку часть из них может быть связана с конкретным подразделением, проектом или типом документа.

В результате анализа было установлено, что исходный набор данных требует предварительной очистки и нормализации, но при этом содержит достаточно информативные тексты для построения классификационного пайплайна. Основные ограничения связаны с неоднородной длиной документов, наличием служебных фрагментов и потенциальной потерей информации при обработке длинных писем transformer-моделями. Эти факторы были учтены при проектировании этапов предобработки, аугментации, суммаризации и классификации.

2.3 Обнаружение дубликатов, пересечений и аномальных объектов

На этапе подготовки данных была выполнена проверка набора писем на наличие дубликатов, пересечений между выборками и аномальных объектов. Для задачи классификации это является важным этапом, поскольку качество итоговой модели должно

оцениваться на документах, которые не использовались при обучении. Если одинаковые или почти одинаковые тексты оказываются одновременно в обучающей и тестовой выборках, итоговые метрики могут быть завышены и не будут отражать реальную способность модели классифицировать новые письма.

В первую очередь были проанализированы точные дубликаты текстов. Под точным дубликатом в данном случае понимается полное совпадение содержимого письма после базовой нормализации. Такие объекты могут появляться из-за повторной загрузки данных, технического объединения нескольких файлов или повторного сохранения одного и того же письма. Наличие точных дубликатов внутри обучающей выборки не всегда приводит к утечке данных, но может усиливать вклад отдельных документов и искажать распределение классов. Поэтому такие случаи необходимо учитывать при дальнейшей обработке.

Кроме дубликатов, анализировались аномальные объекты. К ним относились письма с очень малым количеством содержательного текста, документы с чрезмерно большой длиной, а также тексты, состоящие преимущественно из технических элементов. Такие объекты могут по-разному влиять на модели. Слишком короткие письма содержат мало признаков для классификации, а слишком длинные могут включать пересланные цепочки, подписи, повторяющиеся блоки и служебные строки, которые создают шум.

При обработке аномальных объектов использовался осторожный подход. Полное удаление всех нестандартных фрагментов может привести к потере полезной информации, так как в деловых письмах значимыми могут быть названия подразделений, должности, номера проектов, аббревиатуры и служебные обозначения. Поэтому основной целью являлось не агрессивное сокращение текста, а устранение явно технических проблем и проверка того, что данные пригодны для дальнейшего обучения моделей.

2.4 Правила очистки и нормализации текстов

После анализа структуры датасета, длины документов и наличия потенциальных аномалий был определён набор правил для очистки и нормализации текстов. Цель этого этапа заключалась в том, чтобы уменьшить влияние технического шума и привести письма к более однородному виду, не удаляя при этом информацию, которая может быть значимой для классификации. Для деловой переписки такой подход особенно важен, поскольку часть формально служебных элементов может быть связана с предметной областью, подразделением или конкретным типом документа.

Основные правила очистки включали приведение регистра, нормализацию пробельных символов, обработку технических фрагментов и унификацию типовых элементов. Для некоторых типов данных применялась замена на специальные токены. Такой подход позволяет сохранить сам факт наличия элемента в письме, но не привязывать модель к конкретному значению. Например, наличие ссылки или адреса электронной почты может быть полезным признаком, однако конкретный URL или конкретный адрес чаще всего не должны определять класс документа.

Отдельно учитывалось, что разные модели по-разному чувствительны к предобработке. Для TF-IDF-признаков очистка помогает уменьшить размер словаря и снизить число случайных n -грамм, возникающих из-за технических строк и уникальных значений. Для BERT-подобных моделей чрезмерная очистка может быть нежелательной, так как она нарушает естественную структуру текста. Поэтому итоговый набор правил должен был сохранять связность письма и не превращать документ в набор искусственных токенов.

2.5 Итоговое состояние данных после предобработки

После применения правил очистки количество объектов сократилось с 1774 до 1755. При этом число классов не изменилось и осталось равным 36, что важно для сохранения исходной постановки задачи. Удалённые объекты относились к технически некорректным или недостаточно содержательным записям и не приводили к исчезновению каких-либо классов из датасета.

Более заметным изменением стало сокращение общего объёма текстов. До очистки суммарный объём корпуса составлял 5 186 469 символов, после очистки — 2 173 226 символов. Таким образом, было удалено более половины символьного объёма корпуса. Это связано с устранением служебных блоков, повторяющихся фрагментов, технических элементов и других частей писем, которые создавали шум для моделей классификации. Распределения длины документов после очистки представлены на рисунках 2.4 и 2.5

При этом предобработка выполнялась аккуратно, без агрессивного стемминга или лемматизации, чтобы не разрушить специфические доменные термины и названия подразделений. В частности, сохранялись ключевые аббревиатуры и буквенно-цифровые обозначения, потенциально влияющие на класс объекта. Такой компромиссный подход обеспечивает баланс между снижением шума и сохранением важной предметной информации.

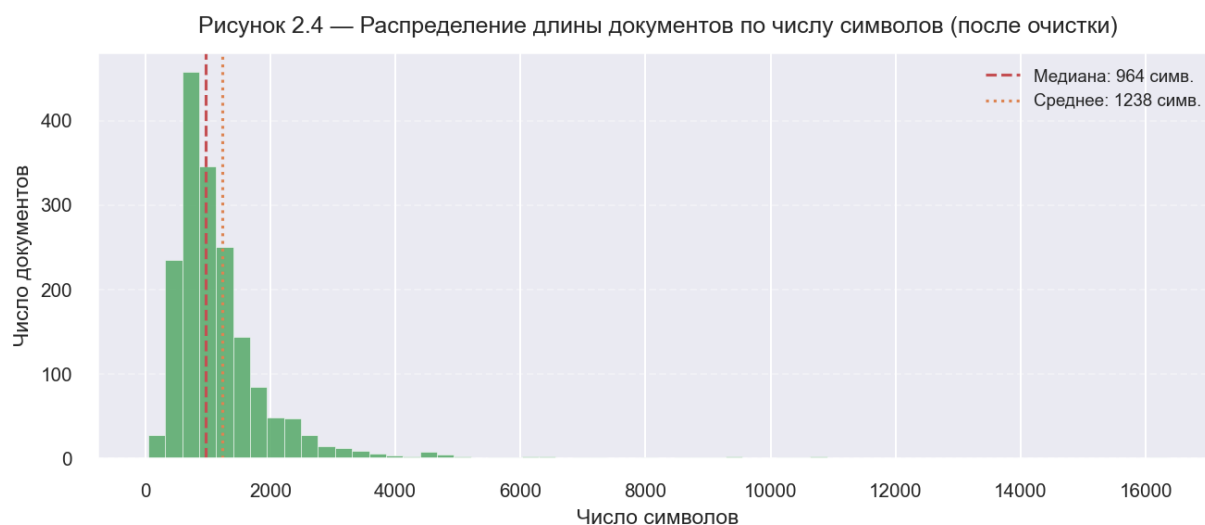


Рисунок 2.4 — Распределение длины документов по числу символов

Рисунок 2.5 — Распределение длины документов (после очистки)

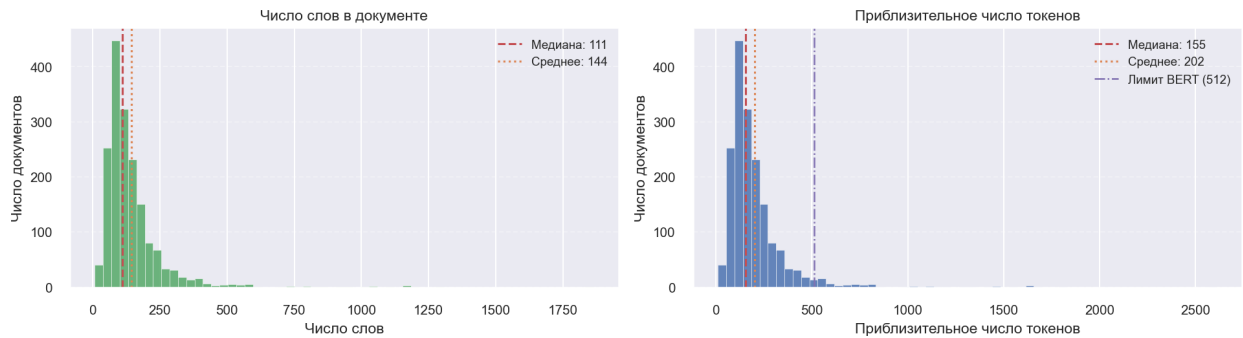


Рисунок 2.5 — Распределение длины документов по числу слов или токенов

2.6 Особенности train/test-разбиения и распределения классов

После очистки и нормализации данных был сформирован итоговый корпус для проведения экспериментов. Он был разделён на обучающую и тестовую выборки. Обучающая часть использовалась для построения признаков, обучения моделей, аугментации и подбора вариантов обработки текста. Тестовая часть сохранялась отдельно и применялась только для итоговой оценки качества классификации.

Таблица 2.1 — Структура train/test-разбиения после очистки данных

Часть выборки	Количество документов	Количество классов
Train	1404	36
Test	351	36
Всего	1755	36

Разбиение выполнялось с сохранением представительности классов, насколько это возможно при имеющемся дисбалансе. Для данной задачи это особенно важно, так как в датасете присутствуют как крупные классы, содержащие сотни документов, так и редкие классы, представленные единичными или малочисленными примерами. Если при разделении данных какой-либо редкий класс исчез бы из тестовой выборки, итоговая оценка не отражала бы качество классификации по полному множеству классов. На рисунках 2.6, 2.7 проиллюстрировано финальное распределение классов.

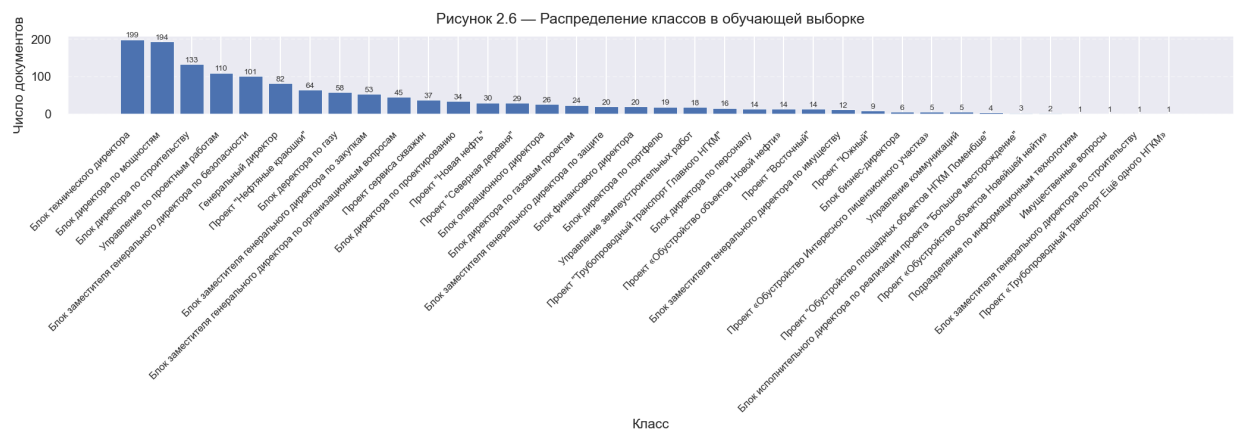


Рисунок 2.6 — Распределение классов в обучающей выборке

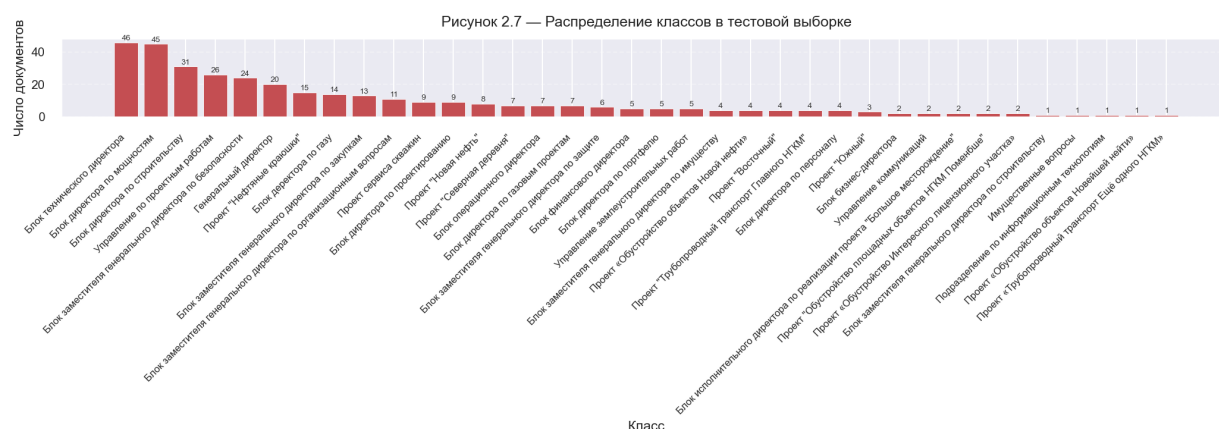


Рисунок 2.7 — Распределение классов в тестовой выборке

Несмотря на сохранение всех классов в обеих частях, распределение остаётся несбалансированным. Крупные категории представлены значительно большим числом документов, чем редкие. Такая структура соответствует реальному характеру деловой переписки: по одним направлениям поток писем формируется постоянно, тогда как другие проекты или подразделения встречаются заметно реже. Поэтому дисбаланс не рассматривался как ошибка данных, а учитывался как одно из условий задачи.

Особенность выбранного разбиения заключается также в том, что тестовая выборка не изменялась на последующих этапах работы. Аугментация, суммаризация обучающих представлений и другие способы расширения данных применялись только к train-части. Это позволило избежать утечки информации из теста в обучение и сохранить корректность сравнения разных моделей. Все методы оценивались на одной и той же тестовой выборке, что делает результаты сопоставимыми между собой.

3 Постановка задачи и проектирование решения

3.1 Формальная постановка задачи многоклассовой классификации

В рамках работы требуется построить программный пайплайн, который по тексту делового письма автоматически определяет его принадлежность к одному из заданных классов. На вход системе подаётся текст документа после этапа предобработки, на выходе формируется метка класса. Предсказание выполняется в режиме *single-label classification*: для каждого письма выбирается только одна итоговая категория.

Задача рассматривается в контролируемой постановке. Для обучающих объектов известны правильные метки, поэтому модели обучаются на парах «текст — класс». После обучения классификатор должен обобщать выявленные закономерности на новые письма, которые не использовались при настройке модели. Все сравниваемые подходы решают одну и ту же задачу, но отличаются способом преобразования текста в признаки и типом классификационной модели.

С инженерной точки зрения задача может быть представлена как последовательность этапов: подготовка текста, построение признакового представления, обучение классификатора и получение предсказания. В качестве признакового представления могут использоваться разреженные TF-IDF-векторы, плотные BERT-эмбединги, суммаризованные варианты текста или композитные векторы. Такой подход позволяет не привязывать постановку задачи к одному конкретному алгоритму и сравнивать несколько решений в едином формате.

3.2 Требования к разрабатываемому решению

Разрабатываемое решение должно обеспечивать полный цикл обработки текстов: от подготовки исходных писем до обучения моделей и сравнения результатов классификации. Поскольку задача относится к прикладной классификации деловой переписки, решение должно быть не только экспериментальным, но и воспроизводимым. Это означает, что для каждого этапа должны быть зафиксированы входные данные, правила обработки, параметры моделей и используемые метрики качества.

3.3 Влияние предметной области на проектирование решения

Предметная область оказывает существенное влияние на выбор методов обработки и классификации текстов. В рассматриваемой задаче документы это длинные деловые письма, связанные с проектами, подразделениями и управленческими направлениями. Для деловой переписки важны как точные лексические совпадения, так и общий контекст письма. Отдельной особенностью является тематическая близость классов. Письма из разных категорий могут использовать похожий деловой язык: согласование, направление документов, контроль сроков, подготовка материалов, организационные вопросы.

3.4 Общая архитектура пайплайна обработки и классификации текста

Разрабатываемое решение построено как модульный пайплайн обработки и классификации текстов. Такой подход был выбран для того, чтобы разделить экспериментальную часть на независимые этапы и упростить воспроизводимость результатов. Каждый крупный этап работы реализован отдельным скриптом или группой скриптов: подготовка данных, очистка, аугментация, суммаризация, построение

признаков, обучение моделей и расчёт метрик не смешиваются в одном файле, а выполняются как самостоятельные части общего процесса.

На первом этапе выполняется подготовка исходных данных. В этот блок входит чтение исходного файла, первичная очистка, нормализация текстов, проверка структуры данных и формирование итогового набора для экспериментов. Отдельная реализация этого этапа позволяет при необходимости повторно подготовить корпус без запуска обучения моделей. Это важно, так как очистка данных влияет на все последующие ветки экспериментов.

На втором этапе формируются дополнительные варианты обучающих данных. В отдельные модули вынесены методы аугментации, их общие части и специфичные под перефразирование и обратный перевод. Такой подход позволяет независимо запускать разные сценарии обогащения выборки и сравнивать их влияние на последующие модели. Для длинных документов в этих модулях дополнительно реализована логика разбиения текста на фрагменты, генерации вариантов и последующей сборки текста обратно в полный документ.

Отдельной веткой пайплайна является суммаризация. Она используется для построения сокращённых версий документов и комбинированных представлений, где исходный текст дополняется его кратким содержанием. Выделение суммаризации в самостоятельный блок позволяет рассматривать её не как часть очистки, а как отдельный способ формирования альтернативного представления текста для классификации.

На следующем этапе выполняется построение признаковых представлений. Классическая ветка пайплайна отвечает за формирование TF-IDF-признаков на основе word и char ngram-представлений. Нейросетевая ветка включает работу с BERT-подобными моделями: получение эмбедингов, эксперименты с различными вариантами классификационных голов, а также обучение модели для длинных документов с использованием чанкинга. Отдельно реализована ветка композитных признаков, в которой объединяются TF-IDF-векторы и BERT-представления.

После построения признаков выполняется обучение классификаторов. Для разных типов представлений используются соответствующие модели: LinearSVC и LogisticRegression для разреженных TF-IDF-признаков, классические модели и MLP для эмбединговых и композитных векторов, а также нейросетевые классификаторы на основе ruBERT и ruRoberta-large. Благодаря модульной структуре можно запускать отдельные эксперименты независимо и не пересчитывать те этапы, которые уже были выполнены.

В общем виде пайплайн можно представить как последовательность связанных, но независимых блоков: подготовка данных → предобработка → аугментация и суммаризация → построение признаков → обучение моделей → оценка качества. Такая архитектура делает решение расширяемым: при добавлении нового метода достаточно реализовать отдельный модуль и подключить его к уже существующим данным и протоколу оценки. Это особенно важно для исследовательской работы, где в процессе экспериментов проверяется несколько альтернативных подходов к представлению и классификации текстов.

3.5 Выбор направлений моделирования

Первым направлением стали классические модели на основе TF-IDF-признаков. Этот подход был выбран как базовая линия, поскольку он хорошо подходит для задач классификации текстов на небольших и средних выборках, быстро обучается и позволяет использовать как словные, так и символьные n-граммы. В работе отдельно рассматриваются word-признаки, char-признаки и их объединение. Поверх полученных векторов обучаются линейные классификаторы, в том числе LinearSVC и LogisticRegression.

Вторым направлением стали BERT-подобные модели. Они были выбраны для проверки того, насколько контекстные представления помогают в задаче классификации длинных русскоязычных деловых писем. В рамках этого направления рассматриваются ruBERT и ruRoberta-large, а также разные варианты использования выхода модели: классификация с использованием AutoModel, mean pooling и полноценный классификатор для длинных документов с чанкингом.

Третьим направлением стало построение эмбединговых представлений документов. В отличие от end-to-end классификации BERT, здесь модель используется для получения векторов текстов, после чего поверх этих векторов обучаются отдельные классификаторы. Такой подход позволяет отделить этап построения признаков от этапа классификации и проверить, насколько информативными являются плотные представления для данной предметной области.

Четвёртым направлением стали композитные признаки. В этом варианте разреженные TF-IDF-векторы объединяются с плотными BERT-представлениями. Идея заключается в том, чтобы совместить точные лексические признаки, полезные для деловых текстов, и контекстную информацию, извлекаемую нейросетевыми моделями. Для классификации таких объединённых векторов рассматриваются классические модели и MLP-классификатор.

3.6 Использование суммаризации для подготовки дополнительных представлений текста

Использование суммаризации позволяет проверить гипотезу о том, что сокращённая версия письма может быть более компактным и менее шумным представлением документа. Если суммаризация сохраняет ключевую информацию, классификатор может получать более сфокусированный текст, в котором меньше повторов и технических элементов. Это особенно важно для BERT-подобных моделей, у которых есть ограничение на длину входной последовательности.

В работе суммаризация рассматривается как дополнительный этап подготовки данных, а не как замена основной предобработки.

3.7 Построение суммаризованного и комбинированного наборов данных

Для проверки влияния суммаризации были сформированы дополнительные варианты набора данных. Первый вариант основан только на суммаризованных текстах. В этом случае вместо полного письма в модель передаётся его краткое содержание. Такой набор позволяет проверить, достаточно ли summary для сохранения признаков, необходимых для определения класса.

Второй вариант является комбинированным. В нём исходный текст объединяется с его суммаризованной версией. Такой подход выбран для того, чтобы не терять детали оригинального письма, но одновременно добавить к нему более компактное представление основного содержания. Комбинированный вариант можно рассматривать как способ обогащения текста: модель получает и полный документ, и его краткое смысловое описание.

При построении таких наборов важно сохранять соответствие между текстом и исходной меткой класса. При этом тестовая часть не используется для обучения моделей суммаризации или настройки классификаторов. Все варианты данных затем оцениваются в едином экспериментальном протоколе.

3.8 Выбор метрик качества классификации

Для оценки качества моделей в работе используются метрики, подходящие для многоклассовой классификации при несбалансированном распределении классов. В такой задаче недостаточно ориентироваться только на ассурасу, поскольку высокая общая точность может достигаться за счёт правильной классификации крупных классов, при этом качество на редких категориях может оставаться низким.

Основными метриками выбраны *balanced accuracy* и *macro F1*. *Balanced accuracy* позволяет учитывать качество распознавания по каждому классу и снижает влияние доминирующих категорий. *Macro F1* также важна для данной задачи, так как рассчитывается с равным вкладом каждого класса и показывает, насколько устойчиво модель работает не только на крупных, но и на малых категориях.

Выбор таких метрик связан с особенностями исходной задачи. Поскольку в данных присутствуют классы с разным количеством документов, итоговая оценка должна показывать не только способность модели распознавать наиболее частые категории, но и её устойчивость на всём множестве классов. Поэтому в финальном сравнении основное внимание уделяется *balanced accuracy* и *macro F1*, а остальные показатели используются для дополнительной интерпретации результатов.

4 Аугментация данных и обогащение выборки

4.1 Мотивация аугментации при дисбалансе классов

Как показал разведочный анализ данных, присутствуют классы, которые содержат всего 1 пример в тренировочной выборке. Есть и менее критичные примеры, но без аугментации у моделей не будет шансов определять класс для таких документов.

4.2 Аугментация классическими методами без использования нейросетевых моделей

Случайное удаление слов, замена синонимами, случайная перестановка слов и другие методы не показали улучшения метрик на методах классификации и в итоговый пайплайн не вошли.

4.3 Общая схема нейросетевой аугментации длинных текстов

Основная аугментация в работе выполняется двумя нейросетевыми методами: перефразированием и обратным переводом. Несмотря на различие самих моделей, общая схема обработки для этих методов имеет сходную структуру. Это позволяет применять единый подход к длинным деловым письмам и контролировать качество получаемых текстов.

Сначала в тексте маскируются чувствительные и структурно важные фрагменты, такие как персоналии, организации и даты. Для этого используются специальные placeholder-обозначения. Маскирование необходимо для того, чтобы генеративная модель не искажала такие элементы и не создавала некорректные варианты имён, организаций или временных выражений.

Затем документ разделяется на фрагменты. Полученные тексты с разными гиперпараметрами подаются нейросетевым моделям. Происходит генерация нескольких примеров с выбором лучшего из них. Полученные новые части документа конкатенируются обратно. Происходит обратный процесс восстановления масок.

Получившееся аугментация сравнивается на схожесть со всеми оригинальными документами класса. Если проверка пройдена пример добавляется, иначе алгоритм переключается на следующий оригинальный документ этого же класса для аугментации. Если не удалось за фиксированное число повторений составить ни одного нового примера, то алгоритм переключается на следующий класс. Это нужно чтобы не заикливаться на сложных примерах и вернуться к ним позже, после того как будут обработаны все простые примеры.

4.4 Разбиение текста на предложения и фрагменты при аугментации

Для применения нейросетевой аугментации длинный документ сначала разбивается на более короткие части. На первом шаге используется разбиение текста на предложения. Такой подход позволяет работать с естественными смысловыми фрагментами, а не с произвольными отрезками текста. Это особенно важно для деловых писем, где одно предложение может содержать отдельное поручение, просьбу, описание ситуации или ссылку на документ.

Однако в деловой переписке встречаются длинные предложения, которые могут превышать допустимую или удобную длину для генеративной модели. Поэтому после разбиения на предложения выполняется дополнительное деление длинных предложений на фрагменты. В реализации используется рекурсивное разбиение по пунктуации, а при

необходимости — по словам. Целевая длина фрагмента ограничивается, чтобы текст оставался удобным для обработки моделью.

Если предложение невозможно корректно разделить по пунктуации, применяется аварийное разбиение на окна фиксированного размера. Такой механизм нужен для устойчивости пайплайна: даже нестандартные или плохо структурированные фрагменты письма должны быть обработаны без остановки всей аугментации.

В результате каждый исходный документ представляется как последовательность фрагментов. Каждый фрагмент аугментируется отдельно, после чего итоговые варианты объединяются обратно. Это позволяет применять перефразирование и обратный перевод к длинным письмам, не теряя структуру документа полностью.

4.5 Генерация нескольких вариантов аугментации и выбор итогового кандидата

Для каждого фрагмента текста генерируется не один, а несколько вариантов аугментации. Это необходимо потому, что результат генеративной модели может отличаться по качеству: один вариант может быть слишком близким к исходному тексту, другой — слишком свободным, третий — содержать неудачную формулировку. Генерация нескольких кандидатов позволяет выбрать наиболее подходящий вариант для включения в итоговый документ.

В методе обратного перевода используются несколько языков-посредников и разные режимы генерации. Это даёт набор альтернативных формулировок для одного и того же исходного фрагмента. В методе перефразирования также используются несколько режимов генерации, включая beam search и sampling. За счёт этого модель может создавать варианты с разной степенью близости к исходной формулировке.

Выбор кандидата выполняется по набору критериев. На первом уровне проверяется техническая пригодность текста: кандидат не должен быть пустым, чрезмерно коротким или полностью совпадать с оригиналом. Далее оценивается близость к исходному фрагменту. Для этого используется косинусное сходство между эмбедингами исходного и сгенерированного текста. Подходящий кандидат должен сохранять смысл исходного фрагмента, но при этом не быть его точной копией.

Для обратного перевода дополнительно используется оценка языковой естественности через перплексию языковой модели. Это помогает выбирать более гладкие и грамматически приемлемые варианты. В результате в итоговый текст попадают не все сгенерированные фрагменты, а только те, которые прошли фильтрацию и соответствуют заданным ограничениям.

4.6 Схема доведения редких классов до порогового размера выборки

Аугментация применялась прежде всего для увеличения количества примеров в редких классах. В работе используется схема доведения малых классов до заданного порогового размера. Это означает, что для каждого класса анализируется текущее количество документов, после чего дополнительные примеры генерируются только в том объёме, который необходим для приближения класса к целевому уровню.

В реализации используется целевое значение — 30. Если класс содержит меньше документов, чем заданный порог, для него запускается генерация новых примеров на основе исходных текстов обучающей выборки. При этом исходные документы перебираются циклически, а число попыток ограничивается, чтобы избежать бесконечной генерации для классов, где модель не может получить достаточно качественных кандидатов.

Каждый новый пример проходит несколько фильтров. Проверяется длина текста, отличие от исходного документа, косинусное сходство с оригиналом и соответствие

классовому пространству. Дополнительно контролируется близость кандидата к эмбедингам текстов того же класса. Это позволяет уменьшить вероятность включения примеров, которые формально сгенерированы из исходного документа, но слишком сильно смещены по смыслу.

Такая стратегия позволяет не увеличивать все классы одинаково, а направить генерацию туда, где данных действительно недостаточно. Это особенно важно для несбалансированного датасета, так как полное равномерное размножение всех классов могло бы привести к избыточному росту корпуса без существенной пользы.

Особую ценность насыщения данных представляет автоматизированный пайплайн. Это позволяет воспроизвести все шаги на любом наборе данных с аналогичной структурой. Данная работа имела два формата запуска: студент запускал пайплайн на маскированном датасете, и научный руководитель запускал тот же пайплайн, но уже на датасете с конфиденциальными данными.

4.7 Аугментация с использованием перефразирования

Перефразирование используется для получения новых вариантов текста с сохранением исходного смысла. В этом подходе модель получает фрагмент письма и генерирует альтернативную формулировку на русском языке. Такой метод хорошо подходит для задачи классификации, так как позволяет увеличить разнообразие выражений внутри одного класса без изменения целевой метки.

В работе для перефразирования использовалась модель семейства `ruT5`. Она применялась как генеративная `seq2seq`-модель: на вход подавался исходный фрагмент текста, а на выходе формировался его перефразированный вариант. Использование русскоязычной модели важно для данной задачи, поскольку исходные документы представляют собой деловые письма на русском языке, содержащие специфичные формулировки, названия проектов и служебные конструкции.

4.8 Аугментация методом обратного перевода

Обратный перевод является вторым нейросетевым методом аугментации, использованным в работе. Его идея заключается в том, что исходный русский текст переводится на промежуточный язык, а затем обратно на русский. При таком преобразовании часть формулировок изменяется, но общий смысл должен сохраняться. Это позволяет получить дополнительный вариант текста без ручного составления правил перефразирования.

Для реализации обратного перевода использовались модели машинного перевода семейства `Helsinki-NLP/OPUS-MT`. Перевод выполнялся по нескольким направлениям: с русского языка на промежуточный язык и затем обратно на русский. В качестве промежуточных языков использовались английский, французский и испанский. Такой выбор позволяет получать несколько разных вариантов переформулирования одного и того же фрагмента.

Метод обратного перевода особенно полезен тем, что он меняет структуру фразы более естественным образом, чем простые случайные замены слов. Однако он также может вносить искажения: термин может быть переведён неточно, деловая формулировка может стать менее естественной, а специфичные названия могут быть изменены. Поэтому перед переводом используются `placeholder`-маркеры для важных сущностей, чтобы, например, не получать переводы аббревиатур.

4.9 Фильтрация аугментированных текстов по косинусному сходству

Фильтрация по косинусному сходству используется как основной механизм контроля качества аугментированных текстов. После генерации необходимо убедиться, что новый текст сохраняет смысл исходного документа, но не является его точной копией. Для этого исходный и аугментированный текст переводятся в векторное пространство, после чего между ними вычисляется косинусное сходство.

В работе для получения эмбеддингов используется модель `deepvk/USER2-base`. Эмбеддинги нормализуются, а сходство вычисляется с помощью косинусной меры. Такой подход позволяет оценивать близость не только по совпадению слов, но и по общему смысловому содержанию. Это особенно важно для перефразирования и обратного перевода, где формулировки могут изменяться, но смысл должен оставаться близким к исходному.

Для отбора кандидатов задаются допустимые интервалы сходства. Если сходство слишком низкое, это может означать, что сгенерированный текст ушёл от исходного смысла. Если сходство слишком высокое, текст почти не отличается от оригинала и не добавляет полезного разнообразия. Поэтому в аугментацию включаются только варианты, находящиеся в заданном диапазоне.

Дополнительно используется проверка близости кандидата к текстам того же класса. Это позволяет контролировать не только связь с исходным документом, но и соответствие общей области класса. Такой фильтр особенно важен для редких классов, где ошибочная аугментация может заметно повлиять на обучение модели.

4.10 Склейка аугментированных фрагментов в полный текст документа

Так как аугментация выполняется на уровне отдельных фрагментов, после генерации необходимо восстановить полный текст документа. Для этого выбранные аугментированные фрагменты объединяются в исходном порядке. Такая схема позволяет сохранить общую структуру письма: последовательность абзацев, логика изложения и порядок смысловых частей остаются близкими к оригинальному документу.

После объединения фрагментов выполняется демаскирование `placeholder`-элементов. На этом этапе специальные обозначения, временно использованные для защиты персоналий, организаций и дат, заменяются обратно на исходные значения. Это позволяет сохранить важные структурные элементы письма и избежать искажений, которые могли бы появиться при генерации.

Далее применяется постобработка сгенерированного текста. Она включает очистку лишних пробелов, исправление технических артефактов генерации и приведение текста к формату, совместимому с остальными данными. Такой этап необходим, поскольку при фрагментарной генерации могут появляться небольшие несогласованности на границах частей.

Склейка фрагментов является важной частью пайплайна, так как итоговая модель классификации работает с полным документом. Поэтому результат аугментации должен представлять собой не набор отдельных предложений, а полноценный текст письма. Это позволяет использовать аугментированные данные в тех же моделях, что и исходные документы.

4.11 Формирование итогового аугментированного набора данных

После генерации перефразированных и обратно переведённых документов были сформированы отдельные аугментированные наборы данных. Для каждого метода сохранялся собственный результат, что позволяло анализировать вклад разных способов

аугментации. Отдельно были получены наборы на основе обратного перевода и перефразирования.

Затем результаты разных методов были объединены в общий аугментированный набор. При объединении выполнялась дедупликация по паре «метка класса — нормализованный текст», чтобы избежать попадания одинаковых объектов. После объединения данные перемешивались с фиксированным значением random state, что обеспечивало воспроизводимость дальнейших экспериментов.

Таблица 4.1 — Сформированные наборы данных после аугментации

Набор данных	Количество объектов	Описание
train.csv	1404	Исходная обучающая выборка после очистки
train_backtranslate.csv	1816	Обучающая выборка с обратным переводом
train_paraphrase.csv	1825	Обучающая выборка с перефразированием
train_augmented.csv	2158	Объединённый аугментированный набор данных

Кроме основного аугментированного набора, далее формировались дополнительные варианты для экспериментов с суммаризацией: суммаризованный аугментированный набор и комбинированный вариант, в котором исходный текст объединяется с summary. Эти данные используются в последующих экспериментах как альтернативные представления обучающей выборки.

Важно, что тестовая выборка при формировании аугментированных данных не изменялась. Это позволяет оценивать все модели на одном и том же наборе документов и сравнивать влияние разных вариантов обучения без изменения тестового протокола.

4.12 Анализ распределения данных после аугментации

После формирования аугментированного набора был выполнен анализ распределения классов. Цель этого этапа заключалась в том, чтобы проверить, насколько аугментация изменила структуру обучающей выборки и удалось ли увеличить количество примеров в редких категориях, распределение классов после аугментации представлено на рисунке 4.1. Поскольку генерация выполнялась преимущественно для малых классов, распределение после аугментации должно стать более сбалансированным по сравнению с исходным train-набором.

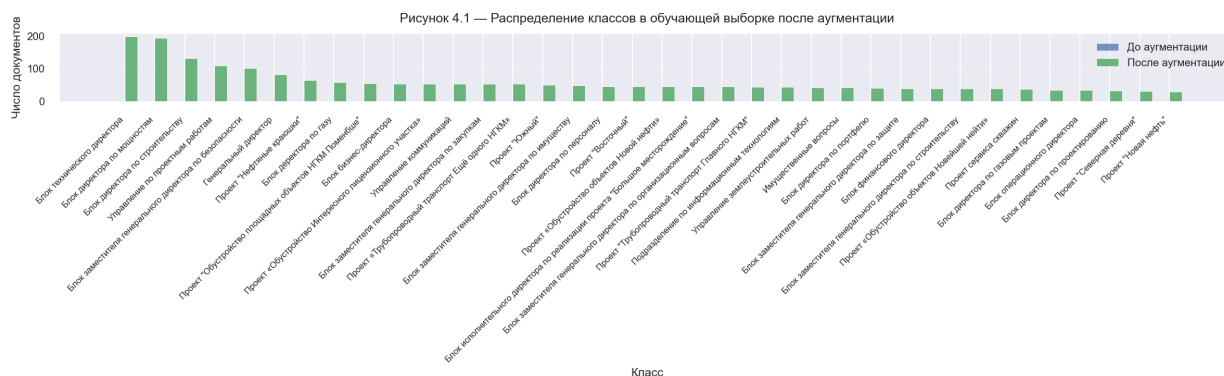


Рисунок 4.1 — Распределение классов в обучающей выборке после аугментации

Анализ показал, что большинство редких классов было увеличено до заданного порогового уровня или приблизилось к нему. При этом крупные классы не размножались пропорционально, что позволило избежать чрезмерного роста уже доминирующих категорий. Таким образом, аугментация была направлена именно на выравнивание обучающего распределения, а не на простое увеличение общего числа документов.

По сравнению с исходной обучающей выборкой объём данных увеличился, при этом все документы сохраняют одну из исходных 36 меток классов. Это позволяет использовать аугментированный набор в тех же моделях классификации, что и исходный train-набор.

Итоговый аугментированный набор далее используется в экспериментах с классическими моделями, BERT-подходами, эмбединговыми и композитными представлениями. Это позволяет оценить влияние обогащения данных на разные типы классификаторов в едином сравнительном протоколе.

5 Классические модели на основе разреженных текстовых признаков

5.1 Теоретические основы TF-IDF-представления текста

TF-IDF является одним из базовых способов преобразования текста в числовое признаковое пространство. В таком представлении документ описывается через набор токенов или n-грамм, а каждому признаку сопоставляется вес. Чем чаще признак встречается в конкретном документе и чем реже он встречается во всём корпусе, тем более информативным он считается для данного документа.

Для задачи классификации деловых писем TF-IDF-подход имеет несколько преимуществ. Во-первых, он хорошо работает с небольшими и средними по размеру наборами данных, где сложные нейросетевые модели могут не получить достаточного количества примеров для устойчивого дообучения. Во-вторых, TF-IDF позволяет явно учитывать слова, словосочетания, служебные обозначения и характерные фрагменты, которые могут быть связаны с конкретным классом. В деловой переписке такими признаками могут быть названия проектов, подразделений, типовые формулировки, внутренние аббревиатуры и устойчивые конструкции.

В отличие от плотных эмбедингов, TF-IDF формирует разреженное пространство признаков. Это означает, что каждому документу соответствует вектор большой размерности, где большинство значений равно нулю. Такая структура естественна для текстов, поскольку один документ содержит только небольшую часть всех возможных слов и n-грамм корпуса. При этом линейные классификаторы хорошо приспособлены к работе с такими признаками и позволяют быстро обучаться даже на больших разреженных матрицах.

В данной работе TF-IDF используется как основа классической ветки моделирования. На его базе строятся признаки разных типов: словные n-граммы, символные n-граммы и их объединение. Такой набор вариантов позволяет проверить, какие уровни текстовой информации оказываются наиболее полезными для классификации длинных русскоязычных деловых писем.

5.2 Линейные классификаторы для разреженных текстовых признаков

Для классификации TF-IDF-признаков в работе используются линейные модели. Их выбор связан с тем, что они хорошо работают с высокоразмерными разреженными представлениями и обычно дают устойчивый результат в задачах классификации текстов. Линейная модель обучает вес для каждого признака и на основе взвешенной суммы признаков принимает решение о принадлежности документа к классу.

Основными классификаторами в классической ветке являются LinearSVC и LogisticRegression. LinearSVC основан на линейной версии метода опорных векторов и хорошо подходит для задач, где данные представлены большим количеством разреженных признаков. LogisticRegression также является линейной моделью, но интерпретирует задачу через оценку вероятностей классов. Оба подхода позволяют эффективно работать с TF-IDF-представлениями и быстро обучаются на CPU.

С учётом дисбаланса классов в обеих моделях учитывается различие в количестве объектов по классам, чтобы уменьшить смещение модели в сторону наиболее крупных категорий. Это особенно важно для рассматриваемой задачи, где часть классов представлена существенно меньшим числом документов.

5.3 Построение признаков на основе word n-грамм

Первый вариант TF-IDF-представления строится на основе словных n-грамм. В этом случае признаками являются отдельные слова и последовательности из нескольких слов. Такой подход позволяет учитывать не только наличие конкретных терминов, но и устойчивые словосочетания, которые могут быть характерны для определённого класса.

В признаковое пространство включаются униграммы и биграммы. Униграммы отражают отдельные слова, а биграммы позволяют учитывать короткие устойчивые сочетания, например названия направлений, типовые деловые обороты или фразы, связанные с конкретной категорией.

5.4 Построение признаков на основе символьных char n-грамм

Второй вариант TF-IDF-представления строится на основе символьных n-грамм. В отличие от word-признаков, здесь единицами анализа являются не слова, а фрагменты строк фиксированной длины. Такой подход полезен для русского языка, поскольку позволяет частично учитывать морфологические изменения слов, окончания, приставки, сокращения и варианты написания.

Символьные n-граммы строятся внутри границ слов. Это помогает уменьшить количество случайных признаков, возникающих при построении n-грамм через пробелы или границы разных слов, и делает признаки более устойчивыми.

Для ограничения словаря исключаются слишком редкие символьные фрагменты, которые встречаются только в единичных документах и могут быть шумом. А также исключаются слишком частые признаки, присутствующие почти во всём корпусе и слабо помогающие различать классы.

Символьные признаки особенно полезны в деловой переписке, где встречаются аббревиатуры, служебные обозначения, проектные названия и разные формы одних и тех же слов. Даже если полные слова отличаются, их символьные фрагменты могут частично совпадать и давать модели дополнительную информацию для классификации.

5.5 Комбинирование word- и char-признаков в едином признаковом пространстве

Поскольку word- и char-признаки отражают разные свойства текста, в работе используется их объединение в одном признаковом пространстве. Word n-граммы хорошо фиксируют отдельные слова и короткие словосочетания, а char n-граммы позволяют учитывать внутреннюю структуру слов и устойчивы к морфологическим изменениям. Для русскоязычных деловых писем такое сочетание является полезным, так как в данных одновременно присутствуют предметные термины, аббревиатуры, служебные обозначения и разные формы слов.

Для объединения признаков включаются два независимых векторизатора: word TF-IDF и char TF-IDF. Каждый из них строит собственное признаковое пространство, после чего полученные матрицы объединяются в общий разреженный вектор документа.

В итоговом пайплайне объединённое представление передаётся в классификатор как единый набор признаков. Такой подход позволяет модели одновременно использовать точные словные совпадения и более гибкие символьные признаки. Это особенно важно для предметной области, где одни классы могут отличаться конкретными названиями и формулировками, а другие — более размытыми лексическими паттернами.

5.6 Реализация пайплайна TF-IDF + LinearSVC

Основным классическим baseline в работе является пайплайн TF-IDF + LinearSVC. Он реализован как последовательность этапов: построение TF-IDF-признаков и обучение линейного классификатора. Такой подход выбран как сильная и вычислительно эффективная базовая линия для задачи многоклассовой классификации текстов.

В качестве признакового пространства используется объединение word- и char-признаков. Word-компонента строится на униграммах и биграммах, а char-компонента — на символьных n-граммах длины от 3 до 5. Далее объединённый разреженный вектор передаётся в LinearSVC.

Использование сбалансированных весов классов связано с неравномерным распределением документов в обучающей выборке. В этом режиме модель учитывает частоты классов и увеличивает вклад редких категорий при обучении. Это позволяет сделать baseline более устойчивым к дисбалансу.

Данный пайплайн имеет несколько практических преимуществ. Он быстро обучается на CPU, не требует GPU и хорошо масштабируется на разреженные признаки. Кроме того, его удобно использовать как контрольную точку для сравнения с более сложными подходами: BERT-моделями, эмбединговыми классификаторами и композитными представлениями.

5.7 Реализация пайплайна TF-IDF + LogisticRegression

Второй линейный классификатор, использованный в классической ветке, — LogisticRegression. Он применяется поверх того же TF-IDF-представления, что и LinearSVC, что позволяет сравнить влияние классификационной модели при одинаковом способе построения признаков.

LogisticRegression полезна как альтернативная линейная модель, поскольку она отличается от LinearSVC способом оптимизации и интерпретацией выходов. В то время как LinearSVC ориентирован на построение разделяющих гиперплоскостей с максимальным зазором, LogisticRegression обучается в вероятностной постановке. Это делает её удобной дополнительной моделью для проверки устойчивости TF-IDF-признаков.

В рамках работы LogisticRegression рассматривается не как замена основному baseline, а как дополнительный классический классификатор. Его использование позволяет убедиться, что результаты классической ветки зависят не только от одного выбранного алгоритма, но и от качества самого признакового представления.

5.8 Дополнительные классические модели классификации

Помимо LinearSVC и LogisticRegression, в экспериментах рассматривался MultinomialNB. Это один из традиционных классификаторов для текстовых задач, особенно часто используемый совместно с частотными или TF-IDF-признаками. Его включение в набор экспериментов позволяет получить простую дополнительную точку сравнения для классических методов.

MultinomialNB основан на вероятностной модели распределения признаков по классам. В задачах классификации текстов он может работать быстро и стабильно, особенно когда признаки являются неотрицательными и отражают частоты или веса слов. При этом модель делает более сильные предположения о независимости признаков, чем линейные классификаторы, поэтому она используется как вспомогательный baseline.

В классической ветке также проводились эксперименты с разными вариантами признаков: только word TF-IDF, только char TF-IDF и объединение word+char. Это

позволяет разделить влияние двух факторов: выбранного классификатора и выбранного способа векторизации текста. Такой подход делает сравнение более полным, поскольку одна и та же модель может вести себя по-разному на разных типах признаков.

Отдельно проверялось влияние нормализации текста перед построением TF-IDF-признаков. Однако это не дало никаких улучшений по метрикам, поэтому эта ветка была отброшена.

5.9 Особенности применения TF-IDF к длинным деловым письмам

Применение TF-IDF к длинным деловым письмам имеет как преимущества, так и ограничения. Одним из главных преимуществ является отсутствие жёсткого ограничения на длину входного текста. В отличие от BERT-подобных моделей, которые принимают ограниченное число токенов, TF-IDF может использовать весь текст документа. Это особенно важно для писем, где значимая информация может находиться не только в начале, но и в середине или конце.

Для деловых писем также полезно то, что TF-IDF хорошо фиксирует характерные лексические признаки. Внутренние названия, проектные обозначения, аббревиатуры, типовые фразы и устойчивые словосочетания могут быть напрямую отражены в признаковом пространстве. При использовании word- и char-ngram-представлений модель получает доступ как к полным словам, так и к их фрагментам.

Однако длинные документы также создают сложности. Они могут содержать большое количество служебных и повторяющихся фрагментов: подписи, пересланную переписку, реквизиты, списки приложений, технические строки. Если такие элементы не обработаны на этапе предобработки, они могут увеличивать размер словаря и создавать шумовые признаки. Поэтому качество TF-IDF-подхода во многом зависит от предварительной очистки данных.

В данной работе TF-IDF используется не только как простой baseline, но и как важная часть общего исследования текстовых представлений. Сначала он применяется в классических моделях, затем его признаки используются при построении композитного векторного пространства вместе с BERT-представлениями. Это позволяет рассматривать TF-IDF как самостоятельный метод и как компонент более сложного признакового описания документа.

6 Контекстные представления и классификация на основе BERT-моделей

6.1 Теоретические основы transformer-архитектур

Transformer-архитектуры занимают центральное место в современной обработке естественного языка благодаря механизму self-attention, который позволяет каждому токenu входной последовательности одновременно учитывать все остальные токены. В отличие от рекуррентных сетей, обрабатывающих текст пошагово, transformer строит контекстуализированное представление каждого токена за один проход через стек энкодер-блоков, каждый из которых содержит многоголовочный self-attention и послонную нормализацию. Это даёт принципиальное преимущество: представление слова определяется его реальным контекстом в конкретном предложении, а не усреднённой статистикой употреблений по корпусу.

BERT-подобные модели — это encoder-архитектуры transformer, предварительно обученные на больших текстовых корпусах с целью восстановления замаскированных токенов и определения является ли текст 1 продолжением текста 2. Полученные при этом параметры кодируют широкие языковые знания, включая синтаксические структуры, морфологические закономерности и семантические связи. При дообучении на прикладной задаче эти знания адаптируются к конкретной предметной области, что делает BERT-подходы возможными для использования на ограниченном объёме размеченных данных.

В данной работе transformer-модели применяются к задаче классификации длинных русскоязычных деловых писем. Принадлежность письма к классу нередко определяется не отдельными ключевыми словами, а их сочетанием в контексте: одно и то же слово в разных письмах может указывать на разные направления деятельности. Кроме того, стандартные BERT-подобные модели ограничены фиксированной максимальной длиной входной последовательности, что требует специальных решений для документов, превышающих этот предел. Именно поэтому в работе реализованы как базовые варианты классификации с усечением текста, так и отдельный классификатор с разбиением документа на перекрывающиеся фрагменты.

6.2 Контекстные эмбединги и их отличие от разреженных признаков

TF-IDF-признаки строят разреженное векторное представление, где каждое измерение соответствует конкретному токenu или n-грамме корпуса, а значение отражает частотность с учётом обратной частоты документа. Такое пространство хорошо фиксирует точные лексические совпадения и устойчивые формулировки, однако каждый токен имеет одно фиксированное значение независимо от контекста, а семантическая близость разных фраз остаётся скрытой.

Контекстные BERT-эмбединги формируют плотное векторное пространство, где позиция документа определяется не набором вхождений слов, а выходом глубокой нейронной сети, обработавшей весь контекст. Одно и то же слово в разных окружениях получает разные векторные представления, что позволяет различать значения омонимов и учитывать модальность высказывания. Для деловой переписки это важно: различие между классами может проявляться в специфических словосочетаниях, описании объектов или характере действий, а не в отдельных ключевых словах.

Оба типа признаков взаимодополняют друг друга. Разреженные признаки обеспечивают высокую интерпретируемость и стабильность на точных формулировках; контекстные — способность к обобщению по семантически близким текстам. В данной работе BERT-представления исследуются как самостоятельный подход в нескольких режимах: прямая end-to-end классификация, построение документных эмбедингов для

обучения классических моделей и как компонент композитного признакового пространства.

6.3 Используемые русскоязычные модели

Для работы с русскоязычными текстами были выбраны модели, предобученные на корпусах, включающих русский язык. В экспериментах использовались две основные модели и одна вспомогательная.

DeepPavlov/rubert-base-cased — русскоязычная BERT-модель базового размера. Её регистрозависимая токенизация важна для деловой переписки с аббревиатурами и наименованиями подразделений. Модель применяется в базовых экспериментах с разными классификационными головами при умеренных вычислительных требованиях.

ai-forever/ruRoberta-large — крупная RoBERTa-модель, оптимизированная для русского языка. По сравнению с базовым BERT она имеет большую размерность скрытых состояний, более мощный encoder и обучалась по улучшённому протоколу без задачи предсказания следующего предложения. Эта модель используется во всех ветках работы: базовая классификация без чанкинга, Sentence-BERT-дообучение для получения эмбедингов, построение композитных признаков и основной классификатор длинных документов.

Таблица 6.1 — Используемые BERT- и SBERT-модели

Модель	Назначение
DeepPavlov/rubert-base-cased	Эксперименты с ruBERT-классификацией
ai-forever/ruRoberta-large	Классификация, эмбединги, чанкинг, композитные признаки

6.4 Два подхода к использованию BERT в задаче классификации

В работе реализованы два принципиально разных способа применения BERT-подобных моделей.

Первый подход — использование BERT как генератора документных эмбедингов. Здесь модель преобразует документ в плотный вектор, который затем передаётся в отдельный классификационный алгоритм: линейный классификатор, метод ближайшего соседа или модель на основе косинусного сходства. Такой подход допускает гибкое использование одних и тех же эмбедингов в разных экспериментальных ветках без повторного дообучения encoder. Эмбединги строятся один раз и затем применяются в нескольких экспериментах: обучение классических классификаторов, косинусная классификация, композитные признаковые пространства.

Второй подход — дообучение BERT как end-to-end классификатора. В этом варианте encoder и классификационная голова обучаются совместно по функции потерь классификации. Все параметры модели, включая параметры transformer-блоков, обновляются под конкретную задачу, что потенциально даёт лучшую адаптацию к предметной области. Недостаток — более высокая вычислительная стоимость и риск переобучения на малой выборке. В этом режиме модель оптимизируется непосредственно под целевые классы, тогда как в первом подходе encoder остаётся фиксированным или дообучается отдельно на метрической задаче без прямого доступа к меткам классов.

Оба подхода реализованы применительно к задаче классификации длинных деловых писем, что позволяет сопоставить не только разные модели, но и разные стратегии использования одного и того же transformer-encoder.

6.4.1 Дообучение Sentence-BERT для построения эмбеддингов

Для построения документных эмбеддингов используется подход на основе библиотеки Sentence Transformers. Архитектура представляет собой стек из трёх компонентов: transformer-encoder, слой pooling и нормализация итогового вектора. В качестве encoder используется ruRoberta-large. Pooling выполняется усреднением скрытых состояний всех токенов с учётом маски внимания, что позволяет каждой позиции последовательности вносить пропорциональный вклад в итоговый вектор. Нормализация выходного вектора обеспечивает, что косинусное сходство между документами совпадает со скалярным произведением их эмбеддингов, что упрощает дальнейшую классификацию и ранжирование.

Обучение выполняется на парах текстовых фрагментов, сформированных из обучающей выборки. Документы одного класса образуют позитивные пары, документы разных классов — негативные. Критически важно: в качестве позитивов берутся только пары чанков из разных документов одного класса. Чанки одного и того же документа не образуют позитивные пары, даже если принадлежат одному классу. Это предотвращает тривиальное решение, при котором модель выучивала бы близость соседних фрагментов текста вместо семантической близости содержания независимых писем. Такая стратегия устраняет риск коллапса представлений и заставляет encoder извлекать признаки, общие для всех документов класса, а не локальные текстовые паттерны внутри одного письма.

В качестве функции потерь применяется MultipleNegativesRankingLoss (MNR). Эта функция формулирует обучение как задачу ранжирования: для каждой позитивной пары в батче все остальные примеры батча выступают в роли in-batch негативов. Encoder оптимизируется так, чтобы вектор якорного примера был ближе к вектору позитивного примера, чем к векторам всех негативов в батче. MNR хорошо масштабируется при большом размере батча и не требует явного сэмплинга негативных пар в датасет: негативы формируются автоматически из структуры батча.

При формировании обучающих данных документы предварительно разбиваются на чанки фиксированного размера с перекрытием. Длина чанка соответствует максимальной длине входа модели, перекрытие обеспечивает сохранение контекста на границах фрагментов. Дополнительно формируется глобальный чанк, объединяющий начальную и конечную части документа: это позволяет учесть ключевые фразы, расположенные в разных частях письма (например, тема в начале и конкретная просьба в конце). Пары строятся между чанками разных документов одного класса, что увеличивает объём обучающих данных при ограниченном числе размеченных писем. Такая стратегия согласована с ограничением длины входа модели: каждый чанк укладывается в допустимый предел, а вариативность фрагментов внутри класса помогает модели обобщать на уровне содержания, а не конкретных формулировок.

На инференсе длинный документ разбивается на чанки аналогично обучающему режиму. Каждый чанк кодируется encoder отдельно, после чего векторы агрегируются в единый документный вектор. Реализованы варианты агрегации: усреднение векторов всех чанков, взятие максимума по каждой компоненте и их комбинация (конкатенация усреднённого и максимального векторов с последующей нормализацией). Итоговый вектор нормализуется на единичную длину. Такой режим инференса согласован с режимом обучения: encoder оптимизировался на представлениях чанков, и при инференсе он применяется к чанкам того же размера и типа.

6.4.2 Дообучение BERT как end-to-end классификатора

В end-to-end постановке transformer-encoder дополняется классификационной головкой, и вся модель обучается совместно под минимизацию функции потерь

классификации. Такой подход позволяет адаптировать не только голову, но и контекстные представления encoder к особенностям конкретных классов. Encoder имеет прямой доступ к меткам классов через градиентный сигнал от функции потерь, что даёт возможность перестроить внутренние представления специфически для целевой задачи.

В работе рассматриваются стандартные варианты без чанкинга и специализированная архитектура для длинных документов. Стандартные варианты проверяют базовые конфигурации с разными способами агрегации выхода encoder (токен CLS и mean pooling) при усечении текста до максимальной длины входа. Основным end-to-end подходом является chunked-классификатор на ruRoberta-large, учитывающий несколько частей длинного письма через разбиение на перекрывающиеся фрагменты с двухуровневой агрегацией. Детали реализации описаны в разделах 6.9–6.14.

6.5 Классификация через специальный токен CLS

Первый вариант end-to-end классификации основан на использовании стандартной архитектуры последовательной классификации из библиотеки Hugging Face Transformers. Transformer-encoder дополняется классификационной головой, которая принимает на вход скрытое состояние специального начального токена CLS и возвращает логиты для каждого из целевых классов. Этот токен благодаря механизму self-attention агрегирует информацию со всей входной последовательности и традиционно используется как её сводное представление в задачах классификации последовательностей.

Данный вариант применяется к обоим основным моделям: ruBERT-base-cased и ruRoberta-large. Тексты токенизируются и усекаются до максимально допустимой длины входа. Для документов, превышающих это ограничение, часть текста отбрасывается, что является известным недостатком подхода применительно к длинным письмам. При этом стандартная архитектура требует минимальных изменений относительно базовой реализации HuggingFace и служит отправной точкой для сравнения с более сложными вариантами. Голова представляет собой линейный слой, проецирующий скрытое состояние CLS-токена в пространство логитов классов.

6.6 Классификация через усреднение скрытых состояний токенов (mean pooling)

Альтернативный вариант агрегации выхода encoder — усреднение скрытых состояний всех токенов с учётом маски внимания. В отличие от использования специального токена CLS, mean pooling задействует информацию из каждой позиции последовательности пропорционально её наличию. Это может быть полезно, когда смысловая нагрузка распределена по всему тексту, а не сосредоточена в начале документа или в одном токене.

Реализация включает: передачу текста через encoder, поэлементное умножение скрытых состояний на маску внимания для исключения паддинговых позиций, суммирование по оси последовательности и деление на количество реальных (не паддинговых) токенов. Полученный усреднённый вектор проходит через слой регуляризации (dropout) и линейный классификатор. Архитектура применяется как для ruBERT-base-cased, так и для ruRoberta-large, что позволяет отдельно оценить влияние способа агрегации и выбранного encoder.

Ограничение на длину входа сохраняется так же, как и в стандартной архитектуре: текст усекается, если превышает допустимый предел. Поэтому mean pooling рассматривается как альтернативный способ использования выхода encoder, а не как решение проблемы длинных документов. Преимущество mean pooling — более равномерное использование информации из всех токенов последовательности, что

теоретически должно улучшить обобщение, когда ключевые признаки класса распределены по документу.

6.7 Эксперименты с ruBERT-base-cased и различными классификационными головами

Модель ruBERT-base-cased применяется как базовая русскоязычная BERT-модель с умеренными вычислительными требованиями. На её основе исследуются два варианта классификации — через стандартную голову с CLS-токеном и через mean pooling токенов. Оба варианта обучаются в одинаковых условиях: одно и то же разбиение данных, одни и те же метрики оценки, одинаковый оптимизатор с cosine-режимом планировщика скорости обучения и фазой warmup в начале обучения.

Эти эксперименты выполняют двойную функцию. Во-первых, они служат нейросетевой базовой линией — наименее сложным BERT-подходом, с которым сопоставляются более крупные и сложные модели. Во-вторых, сравнение двух вариантов головы внутри одной модели позволяет изолированно оценить вклад способа агрегации выхода encoder при прочих равных условиях. Если mean pooling даёт заметное улучшение относительно CLS-подхода, это указывает на распределённый характер информативных признаков в текстах; если разница минимальна — на достаточность представления CLS-токена.

Поскольку оба варианта работают с усечением длинных документов, результаты отражают качество классификации при неполной передаче текста и могут недооценивать потенциал BERT-подхода в задаче с длинными письмами. Эти эксперименты закладывают нижнюю границу качества нейросетевых подходов.

6.8 Эксперименты с ruRoberta-large и различными классификационными головами

Модель ruRoberta-large тестируется в тех же архитектурных вариантах (стандартная голова с CLS и mean pooling), что и ruBERT. По сравнению с ruBERT-base-cased эта модель имеет большую размерность скрытых состояний, больше слоёв encoder и более качественно оптимизированные веса предобучения благодаря улучшённому обучающему протоколу без задачи предсказания следующего предложения. Это делает её более мощным encoder, однако требовательнее к вычислительным ресурсам: при тех же ограничениях памяти приходится использовать меньший размер батча или дополнительные техники экономии памяти.

Эксперименты без чанкинга используют усечение текста до максимальной длины входа, что ограничивает объём передаваемой информации для длинных писем. Они служат базовой линией ruRoberta-large перед переходом к chunked-архитектуре и позволяют оценить, насколько более мощный encoder компенсирует потерю информации при усечении. Если ruRoberta-large без чанкинга превосходит ruBERT, это указывает на преимущество более глубоких представлений. Если разница невелика, это подчёркивает критичность ограничения длины входа для данной задачи и обосновывает необходимость chunked-архитектуры.

6.9 Ограничение длины входной последовательности и проблема длинных документов

BERT-подобные модели обрабатывают последовательность токенов фиксированной максимальной длины. Длина определяется архитектурой позиционных

эмбедингов и обычно составляет 512 токенов. Для коротких текстов это ограничение не критично, однако деловые письма могут существенно превышать этот предел. Согласно анализу данных, значительная доля документов в исследуемом корпусе после очистки всё равно содержит больше токенов, чем допускает стандартный вход.

Простое усечение текста до максимальной длины ведёт к потере информации из средней и конечной части письма. Для деловой корреспонденции это особенно нежелательно: ключевой смысловой фрагмент может располагаться в любом месте текста. Если усечение отбрасывает именно ту часть письма, которая определяет класс, модель не сможет корректно классифицировать документ независимо от мощности `encoder`.

Именно поэтому базовые BERT-эксперименты без чанкинга используются лишь как отправная точка, а основное решение строится вокруг явного разбиения документа на перекрывающиеся фрагменты. Такой подход позволяет модели обработать весь документ, не нарушая ограничений архитектуры `transformer`.

6.10 Chunked-классификатор: архитектура для длинных документов

Основной классификатор длинных документов строится поверх `ruRoberta-large` и включает три уровня: `encoder`, механизм обработки чанков с двухуровневой агрегацией и классификационную голову.

Разбиение на перекрывающиеся чанки. Для обработки документов, превышающих ограничение длины входа, текст разбивается на несколько перекрывающихся фрагментов. Каждый чанк формируется из последовательности токенов заданной длины, равной максимально допустимой длине входа модели. Соседние чанки перекрываются на фиксированное число токенов, что реализуется через параметр `stride` при токенизации. Перекрывание гарантирует, что информация на границах фрагментов не теряется: фраза, разделённая двумя чанками при непересекающемся разбиении, полностью попадает хотя бы в один из них при скользящем окне.

Для ограничения вычислительной сложности число чанков на один документ ограничено сверху. Документы, генерирующие больше фрагментов, усекаются до максимального количества чанков. Если реальное число фрагментов меньше максимума, позиции дополняются паддингом — чанками из паддинговых токенов. Маска числа чанков передаётся в модель как дополнительный вход, позволяя модели отличать содержательные фрагменты от пустых. Такая схема сохраняет единообразный размер тензоров в батче при переменной длине документов и позволяет эффективно обрабатывать батчи документов разной длины.

Двухуровневая агрегация представлений. После обработки каждого чанка `encoder`-моделью выполняется двухуровневая агрегация: сначала на уровне токенов внутри чанка, затем на уровне чанков внутри документа.

Первый уровень — токеничная агрегация. Для каждого чанка выполняется `mean pooling` скрытых состояний всех токенов с учётом маски внимания. Скрытые состояния поэлементно умножаются на маску внимания, суммируются по оси последовательности и делятся на количество реальных (не паддинговых) токенов. Таким образом, каждый чанк преобразуется в один вектор фиксированной размерности, отражающий агрегированное содержание данного фрагмента документа. Паддинговые токены исключаются из расчёта, что обеспечивает корректность усреднения для чанков разной реальной длины.

Второй уровень — чанковая агрегация. Векторы всех чанков одного документа усредняются с учётом маски числа фрагментов, исключаящей паддинговые чанки из расчёта. Если документ содержит меньше чанков, чем максимум, паддинговые позиции имеют нулевую маску и не влияют на итоговый вектор. Результат — единый документный вектор, представляющий собой взвешенное среднее по всем реальным чанкам. Такая двухуровневая схема выбрана как простой и устойчивый способ агрегации

без введения дополнительных обучаемых параметров, что снижает риск переобучения на малой выборке.

Encoder применяется независимо к каждому чанку. Входной тензор преобразуется в плоский вид, обрабатывается encoder как батч независимых последовательностей, после чего результаты перегруппировываются обратно в многомерную структуру. Это позволяет эффективно использовать параллельные вычисления GPU для обработки всех чанков документа одновременно.

6.11 Классификационная голова и функция потерь

Классификационная голова состоит из трёх компонентов: нормализация входного вектора, слой регуляризации и линейный выходной слой. Такая упрощённая архитектура LayerNorm → Dropout → Linear выбрана на основе экспериментов: в ранних версиях использовалась более сложная голова Dropout → Linear → GELU → Dropout → LayerNorm → Linear с двумя линейными слоями и промежуточной нелинейностью. Это приводило к нестабильной сходимости и переобучению уже после первых эпох обучения на малой выборке. Упрощение до однослойной схемы значительно улучшило стабильность обучения и финальное качество. Для задач классификации с малым количеством обучающих примеров переусложнённая голова мешает достичь хорошего качества: избыточная параметризация ведёт к запоминанию тренировочной выборки вместо обобщения на validation.

Нормализация перед классификатором стабилизирует масштаб входных активаций, что особенно важно при разных значениях mean pooling по чанкам различной длины и числа. Dropout подавляет зависимость предсказания от отдельных компонент вектора, заставляя модель использовать распределённое представление.

Функция потерь — кросс-энтропия с дополнительными механизмами: весами классов для учёта дисбаланса и сглаживанием меток. Веса классов вычисляются по частоте каждого класса в обучающей выборке методом сбалансированного взвешивания: классы с меньшим числом примеров получают больший вес в функции потерь, что усиливает градиентный сигнал от редких классов. Сглаживание меток предотвращает излишнюю уверенность модели в предсказаниях: вместо целевого вектора с единицей на правильном классе и нулями на остальных используется слегка размытое распределение. Это особенно важно при наличии пограничных документов с признаками нескольких классов и аугментированных текстов, которые могут содержать семантический шум или артефакты генерации.

6.12 Параметры обучения chunked-классификатора

Обучение chunked-классификатора учитывает специфику задачи: крупная модель, малая выборка, несбалансированные классы, длинные документы.

Размер батча и накопление градиентов. Поскольку обработка нескольких чанков одного документа требует значительного объёма видеопамяти, используется размер батча равный одному документу. Чтобы получить эффективный батч достаточного размера для стабильной оценки градиента, применяется накопление градиентов: параметры модели обновляются не после каждого документа, а после обработки нескольких документов с суммированием их градиентов. Это позволяет имитировать обучение на большом батче без превышения лимитов памяти. Эффективный размер батча определяется произведением реального размера (один документ) и числа шагов накопления.

Раздельные группы параметров. В финальной конфигурации скорости обучения для encoder и головы установлены одинаковыми. Раздельная группировка параметров сохранена для избирательного применения weight decay: регуляризация применяется

только к весам линейных слоёв, но не к параметрам нормализации (LayerNorm) и смещениям (bias), критичным для стабилизации обучения.

Планировщик скорости обучения. Используется cosine-режим планировщика с warmup-фазой в начале обучения. Warmup постепенно увеличивает скорость обучения от нуля до целевого значения на протяжении заданной доли обучающих шагов. Это позволяет избежать резких обновлений параметров в первых итерациях, когда голова ещё случайная и может генерировать большие градиенты. После warmup скорость обучения плавно снижается по cosine-кривой до нуля к концу обучения. Такой режим улучшает сходимость и финальное качество модели.

Ограничение нормы градиента. Обучение ведётся с обрезкой градиентов по глобальной норме. Если норма вектора градиентов превышает заданный порог, все градиенты масштабируются пропорционально, чтобы норма не превосходила предел. Это предотвращает взрывной рост градиентов, который может возникнуть при обработке документов с нетипичными формулировками или при резких обновлениях на начальных этапах обучения.

Early stopping и сохранение лучших весов. Обучение останавливается автоматически, если целевая метрика на validation-выборке не улучшается на протяжении нескольких эпох. Это предотвращает переобучение на малой обучающей выборке. Лучшие веса модели (соответствующие максимальному значению целевой метрики на validation) сохраняются в памяти процесса, а не на диске. После завершения обучения эти веса восстанавливаются перед финальной оценкой модели. Такой подход гарантирует, что итоговая модель соответствует пику качества на validation, а не последней эпохе обучения, которая может уже содержать признаки переобучения.

6.13 Регуляризация и стабилизация обучения на малой выборке

Ограниченный объём обучающей выборки при большом числе классов создаёт риск переобучения для крупных параметрических моделей. В chunked-классификаторе применяется комплекс мер для устойчивого обучения.

Dropout в классификационной голове снижает зависимость предсказания от отдельных компонент документного вектора. На каждой итерации обучения случайная доля элементов вектора обнуляется, вынуждая модель строить более распределённые представления. Это предотвращает запоминание конкретных паттернов обучающей выборки и улучшает обобщение на новые документы.

Сглаживание меток не позволяет выходным вероятностям стремиться к единице на правильном классе и нулю на остальных. Целевое распределение размывается: доля вероятностной массы перераспределяется с правильного класса на все остальные равномерно. Это снижает уверенность модели в предсказаниях и препятствует подстройке под отдельные примеры. Особенно важно при наличии аугментированных текстов: они могут содержать семантический шум или артефакты генерации, и излишняя подгонка под них вредит обобщению.

Веса классов в функции потерь усиливают градиентный сигнал от редких классов, которые иначе были бы подавлены численно преобладающими категориями. Этот механизм согласован с применением аугментации: аугментация увеличивает число примеров в малых классах, доводя их до сопоставимого уровня с крупными; веса классов дополнительно подчёркивают важность корректной классификации этих выравненных классов при обучении. Оба механизма вместе обеспечивают более равномерное обучение классификатора по всем категориям, предотвращая доминирование наиболее частых классов в финальной модели.

Gradient checkpointing снижает потребление видеопамяти при обратном проходе через глубокую сеть. Вместо сохранения всех промежуточных активаций для вычисления

градиентов сохраняется только часть контрольных точек, а остальные активации пересчитываются при необходимости. Это позволяет обучать крупный encoder на длинных чанках в доступных вычислительных условиях, хотя и увеличивает время одной итерации обучения. Без gradient checkpointing обучение ruRoberta-large с несколькими чанками на документ невозможно на GPU стандартного объёма памяти.

7 Композитные и эмбединговые представления текста

7.1 Варианты BERT-эмбедингов: предобученные и дообученные

Исследуются два варианта построения эмбедингов, различающиеся наличием адаптации encoder к целевым классам.

Эмбединги предобученной модели строятся прямым применением ruRoberta-large без дополнительного дообучения. В этом случае модель используется как готовый языковой encoder, обученный на широком корпусе русскоязычных текстов. Такой вариант позволяет проверить, насколько общие знания о языке, зафиксированные в предобученной модели, применимы к предметной области деловой переписки без явной адаптации к структуре классов. Преимущество — простота и воспроизводимость: эмбединги вычисляются без этапа обучения. Недостаток — отсутствие специализации: модель может хорошо отражать общую семантическую близость, но не обязательно выделяет признаки, критичные для различения корпоративных писем по управленческим блокам. Эти эмбединги служат baseline плотного представления и позволяют оценить «out-of-the-box» качество encoder на целевой задаче.

Эмбединги дообученной модели получаются после адаптации encoder на данных целевой задачи. Цель такого дообучения — сформировать пространство, в котором документы одного класса располагаются ближе друг к другу, чем документы разных классов. Дообучение выполняется по схеме, описанной в разделе 6.4.1: используются пары текстовых фрагментов из разных документов одного класса, а оптимизация проводится с помощью MultipleNegativesRankingLoss. Такая постановка предотвращает обучение на тривиальной близости перекрывающихся фрагментов одного письма и заставляет encoder извлекать признаки, устойчивые на уровне класса. После дообучения encoder применяется для построения эмбедингов тем же способом, что и предобученная версия: разбиение на чанки, независимое кодирование, агрегация и нормализация.

Сравнение этих вариантов позволяет изолированно оценить вклад адаптации encoder к структуре целевых классов. Если дообученные эмбединги значительно превосходят предобученные, это указывает на важность метрического обучения для данной задачи.

7.2 Классические классификаторы поверх плотных эмбедингов

После построения BERT-эмбедингов задача классификации решается с помощью классических моделей машинного обучения. Каждый документ представлен плотным вектором фиксированной размерности, а классификатор обучается не на текстах, а на этих векторах. Такой подход позволяет отдельно исследовать качество признакового пространства: если эмбединги уже содержат достаточно информации о принадлежности к классам, даже простая линейная модель должна успешно построить разделяющую границу.

В работе применяются LinearSVC и LogisticRegression. Оба классификатора строят линейные разделяющие гиперплоскости в пространстве эмбедингов. Их использование позволяет проверить гипотезу линейной разделимости классов в BERT-пространстве: если метрики сопоставимы с более сложными моделями, это указывает на то, что encoder уже выполнил основную работу по преобразованию текстов в хорошо разделимое пространство. LinearSVC использует SVM с линейным ядром и оптимизируется по hinge loss. LogisticRegression минимизирует логистическую функцию потерь и возвращает калиброванные вероятности классов, что полезно для анализа уверенности модели.

Преимущество этого подхода — вычислительная эффективность. Encoder применяется к данным один раз, после чего эмбединги сохраняются. Обучение классических моделей поверх готовых векторов требует минимальных ресурсов (секунды

на CPU) и позволяет быстро тестировать разные алгоритмы классификации. Кроме того, одни и те же эмбединги используются в нескольких ветках экспериментов, что делает результаты напрямую сопоставимыми: разница в качестве объясняется только выбором классификатора, а не различиями в признаках.

7.3 Классификация по косинусному сходству: центроиды и ближайшие соседи

Помимо обучения классификационных моделей, эмбединги используются в методах прямого сравнения по косинусному сходству. Эти методы не требуют обучения разделяющих границ и напрямую используют геометрию эмбедингового пространства.

Классификация по центроидам вычисляет для каждого класса средний вектор по всем обучающим документам данного класса. Для нового документа строится эмбединг и вычисляется косинусное сходство с центроидами всех классов. Документ относится к классу с наибольшим сходством. Этот метод полезен как диагностика качества пространства: если эмбединги хорошо группируют документы одного класса, центроиды должны быть чётко различимы, а новые документы — притягиваться к центроиду своего класса сильнее, чем к чужим. Ограничение — каждый класс описывается одним усреднённым вектором, что может быть недостаточно для классов с внутренним разнообразием. Если внутри класса существуют несколько подгрупп с разным содержанием, центроид может оказаться далеко от всех реальных документов.

Классификация методом ближайшего соседа для тестового документа находит обучающий документ с максимальным косинусным сходством и присваивает его метку. В отличие от центроидов, метод сохраняет локальную структуру пространства и учитывает конкретные примеры. Это полезно, когда внутри класса существуют разные подгруппы похожих документов: метод найдёт ближайший пример из подходящей подгруппы, даже если центроид класса далёк. Недостаток — чувствительность к шумным объектам в обучающей выборке: один нетипичный документ с неправильной меткой может стать ближайшим соседом и привести к ошибке. Также метод требует вычисления сходства со всеми обучающими примерами, что менее эффективно, чем сравнение только с центроидами.

Оба метода используются как дополнительные способы анализа эмбедингового пространства, а не как основные классификаторы. Их результаты показывают, насколько документы одного класса сконцентрированы в пространстве и как чётко разделены классы между собой. Если центроидный метод даёт высокое качество, это указывает на компактность классов; если ближайший сосед значительно лучше центроидов — на наличие подгрупп внутри классов.

7.4 Конкатенация TF-IDF и BERT-векторов

Разреженные TF-IDF-признаки и плотные BERT-эмбединги отражают разные свойства текста. TF-IDF хорошо фиксирует точные лексические совпадения: конкретные термины, аббревиатуры, внутренние обозначения подразделений, характерные фразы деловой переписки. Если два письма содержат одну и ту же формулировку (например, название проекта или код департамента), TF-IDF зафиксирует это как прямое совпадение соответствующих компонент вектора. BERT-эмбединги, напротив, позволяют учитывать контекст и семантическую близость, различая значения одних и тех же слов в разных окружениях. Они полезны для обобщения по документам с разными формулировками, но схожим содержанием. Поэтому в работе исследуется объединение этих представлений в один композитный вектор.

Конкатенация выполняется на уровне признаков: для каждого документа строится TF-IDF-вектор и BERT-вектор, затем они объединяются в одно общее пространство. В результате классификатор получает одновременно разреженные лексические признаки и

плотные контекстные признаки, что потенциально позволяет использовать преимущества обоих подходов. Для деловых писем это особенно важно: класс может определяться как точными маркерами (название проекта, код подразделения, устойчивые фразы), так и общим содержанием документа (описание действий, запросов, характер обсуждения).

7.5 Реализация композитного признакового пространства

Композитное признаковое пространство строится в несколько этапов. Сначала для каждого документа формируется TF-IDF-представление. Word-компонента включает униграммы и биграммы слов, char-компонента — символьные n-граммы длины от трёх до пяти. Эти признаки объединяются в разреженный вектор, фиксирующий лексические паттерны текста.

Параллельно для того же документа строится BERT-эмбеддинг с использованием стратегии разбиения на чанки, описанной в разделе 6.10. Итоговый BERT-вектор имеет фиксированную размерность, соответствующую скрытому пространству ruRoberta-large.

Перед объединением признаки нормализуются независимо: TF-IDF-вектор и BERT-вектор приводятся к единичной норме. Это важно, поскольку они имеют разную природу и разные исходные масштабы значений. После нормализации BERT-компонента дополнительно масштабируется весовым коэффициентом, что позволяет контролировать баланс между разреженными и плотными признаками в итоговом векторе. Коэффициент подбирается эмпирически так, чтобы компенсировать разницу размерностей двух пространств и обеспечить сопоставимый вклад BERT-признаков относительно высокоразмерного TF-IDF-пространства. После масштабирования и конкатенации выполняется финальная L2-нормализация объединённого вектора.

Полученное композитное пространство используется как вход для последующих классификаторов. Его основная идея — не выбирать между классическим и нейросетевым представлением, а объединить их в одном векторе документа, позволяя модели использовать оба типа информации одновременно.

7.6 Классические модели и MLP поверх композитных векторов

После построения композитных векторов поверх них обучаются классификационные модели двух типов: классические линейные модели и MLP-классификатор. Это позволяет проверить, достаточно ли линейной разделимости объединённого пространства или требуется нелинейное преобразование признаков.

Классические модели — LinearSVC и LogisticRegression — применяются к композитному пространству аналогично тому, как использовались в чистых TF-IDF- и BERT-ветках. Такой вариант позволяет оценить, насколько объединение двух типов признаков меняет поведение линейных классификаторов и даёт ли их комбинация преимущество относительно каждого представления по отдельности.

Дополнительно используется MLP-классификатор. В отличие от линейных моделей, он способен учитывать нелинейные взаимодействия между компонентами признакового пространства. Это может быть полезно, поскольку TF-IDF- и BERT-компоненты имеют разную природу: первая отражает точные лексические совпадения, вторая — контекстную семантику. Нелинейная модель потенциально может лучше комбинировать такие признаки, выявляя паттерны их совместного присутствия.

MLP обучается поверх уже сформированных композитных векторов. Это разделяет этап построения признаков и этап классификации, позволяя сравнивать разные классификаторы на одном и том же входном представлении. Такой подход делает экспериментальную схему прозрачнее: изменение качества можно связывать либо с признаковым пространством, либо с выбранной моделью классификации.

7.7 Композитная ветка как объединение классических и нейросетевых подходов

Композитная ветка завершает исследование различных способов представления текста. В ней объединяются результаты классического TF-IDF-подхода и BERT-эмбедингов, после чего проверяется несколько вариантов классификации поверх общего признакового пространства. Такая архитектура позволяет сопоставить вклад разреженных и плотных признаков: если композитное пространство даёт заметное улучшение относительно чистых TF-IDF- или BERT-веток, это указывает на дополняющий характер двух типов представлений.

Основная гипотеза композитной ветки состоит в том, что разные типы признаков могут покрывать разные аспекты классификационной задачи. TF-IDF эффективен для точных маркеров класса — устойчивых формулировок, терминов, названий. BERT-эмбединги полезны для общего содержания и контекстуальных паттернов. Их объединение потенциально позволяет использовать оба механизма одновременно, что особенно важно в задаче с длинными деловыми письмами, где класс может определяться как прямыми упоминаниями проектов и подразделений, так и общим характером описываемых действий и запросов.

8 Программная реализация разработанного пайплайна

8.1 Общая структура программного решения

Программная реализация работы построена как модульный исследовательский пайплайн. Основная идея такой структуры заключается в том, что каждый крупный этап обработки данных и обучения моделей вынесен в отдельный python-модуль. Это упрощает повторный запуск экспериментов, позволяет независимо изменять отдельные части решения и снижает риск смешивания логики подготовки данных, обучения и оценки качеств. Каждая версия эксперимента хранится в git'e вместе с получившимися метриками.

В репозитории выделены несколько основных направлений: подготовка исходного датасета, очистка и нормализация текстов, аугментация данных, суммаризация, классические baseline-модели, BERT-эмбединги, BERT-классификация, классификация по косинусному сходству и построение гибридных признаков. Такая структура соответствует общей логике исследования: сначала формируется единый подготовленный корпус, затем на его основе строятся разные варианты признаковых представлений и обучаются модели.

Ключевым входным этапом является подготовка данных из исходного файла. Для этого используется скрипт `prepare_dataset.py`, который запускает очистку, формирует очищенный датафрейм и создаёт train/test-разбиение. После этого отдельные ветки пайплайна работают уже с подготовленными файлами, такими как `train.csv`, `test.csv` и их аугментированные или суммаризованные варианты.

С инженерной точки зрения решение организовано так, чтобы отдельный эксперимент можно было запускать без полной пересборки всего проекта. Например, классические модели на основе TF-IDF, BERT-классификация и гибридные модели находятся в разных частях репозитория и используют общие подготовленные данные. Это делает код более поддерживаемым и позволяет последовательно расширять экспериментальную часть.

8.2 Модуль предобработки данных

Модуль предобработки отвечает за преобразование исходных писем в очищенный и нормализованный корпус. Основная логика подготовки данных реализована в `prepare_dataset.py` и вспомогательных скриптах каталога `utils/`. Этот этап выполняется до обучения моделей и является общим для всех последующих экспериментальных веток.

В процессе предобработки выполняется чтение исходного набора данных, удаление или нормализация технических фрагментов, проверка структуры данных и сохранение очищенного датафрейма. Для очистки используются регулярные выражения и эвристики, направленные на обработку служебных блоков, повторяющихся фрагментов, списков, реквизитов и других элементов, которые могут создавать шум в признаковом пространстве.

Отдельная часть предобработки связана с формированием train/test-разбиения. После очистки данные делятся на обучающую и тестовую части с сохранением всех классов. Это разбиение далее используется во всех экспериментах, что обеспечивает сопоставимость результатов. Тестовая часть не используется при аугментации и обучении моделей.

Результатом работы модуля предобработки являются подготовленные файлы данных с колонками `text` и `label`. Такой простой формат удобен для дальнейшей работы: классические модели, BERT-пайплайны, аугментация, суммаризация и гибридные подходы могут использовать одинаковую структуру входных данных.

8.3 Модуль аугментации текстов

Модуль аугментации реализован в каталоге `augmentation/` и разделён на несколько логических частей. Общие функции вынесены в подкаталог `common/`, а конкретные методы аугментации расположены в отдельных ветках `paraphrase/` и `backtranslate/`. Такое разделение позволяет использовать общую инфраструктуру обработки длинных текстов для разных способов генерации новых примеров.

Общая логика аугментации включает маскирование значимых сущностей, нормализацию текста, разбиение документа на предложения и фрагменты, генерацию нескольких кандидатов, фильтрацию по качеству и последующую сборку полного документа. Такой подход необходим из-за большой длины деловых писем: генерация нового текста целиком была бы менее устойчивой и сильнее зависела бы от ограничений используемых моделей.

В ветке перефразирования используется генеративная `ruT5`-модель, создающая альтернативные формулировки русскоязычных фрагментов. В ветке обратного перевода применяются модели машинного перевода семейства `Helsinki-NLP/OPUS-MT`, где русский текст переводится на промежуточный язык и обратно. Для обоих методов реализована фильтрация кандидатов по косинусному сходству и дополнительная постобработка сгенерированного текста.

Отдельно реализована логика доведения редких классов до заданного порогового размера. Генерация новых примеров выполняется преимущественно для тех классов, где исходных документов недостаточно. Итогом работы модуля становятся отдельные аугментированные наборы, например `train_paraphrase.csv`, `train_backtranslate.csv`, а также объединённый `train_augmented.csv`.

8.4 Модуль построения векторных представлений

Построение векторных представлений реализовано в нескольких частях проекта, поскольку в работе используются разные типы признаков. Классические разреженные признаки строятся в `baseline`-ветке на основе `TfidfVectorizer`. Для этого используются `word n`-граммы, `char n`-граммы и их объединение через `FeatureUnion`.

Для BERT-представлений используется каталог `bert_embeddings/`. В нём реализовано построение плотных эмбеддингов документов на основе `ai-forever/ruRoberta-large`. Так как письма могут быть длинными, используется отдельный механизм обработки длинных текстов: документ разбивается на чанки, каждый чанк кодируется моделью, после чего представления агрегируются в единый вектор документа.

Отдельная часть построения признаков относится к композитным векторам. Она реализована в каталоге `hybrid/`. В этой ветке для одного и того же документа строятся TF-IDF-признаки и BERT-эмбеддинг, после чего они объединяются в общее признаковое пространство. Перед объединением выполняется нормализация, а вклад BERT-компоненты может дополнительно масштабироваться.

Такая организация позволяет использовать разные представления независимо друг от друга. Один и тот же подготовленный текст может быть преобразован в разреженный TF-IDF-вектор, плотный BERT-вектор или композитный вектор. Это обеспечивает единую основу для сравнения разных подходов к классификации.

8.5 Модуль обучения моделей

Обучение моделей также разделено на несколько независимых веток. Классические модели реализованы в `baseline.py` и скриптах каталога `baseline_experiments/`. В этих

экспериментах проверяются разные варианты TF-IDF-признаков и классификаторов, включая LinearSVC, LogisticRegression и MultinomialNB.

BERT-классификация реализована в каталоге `bert_classification/`. В нём выделены отдельные файлы для конфигурации, подготовки данных, модели, обучения, инференса и расчёта метрик. Основной классификатор построен на основе `ai-forever/ruRoberta-large` и поддерживает обработку длинных документов через разбиение на чанки. Для обучения используется MLP-голова, веса классов, `label smoothing` и разные `learning rate` для `encoder` и классификационной головы.

Ветка эмбедингов и Sentence-BERT-подхода реализована в `bert_embeddings/`. Она отвечает за обучение модели представлений и последующее получение эмбедингов документов. Эти эмбединги далее могут использоваться в классических классификаторах, методах косинусной классификации и гибридных моделях.

Методы классификации по косинусному сходству вынесены в каталог `cosine_similarity_classification/`. Там реализованы подходы на основе центроидов классов и ближайшего соседа. Гибридные классификаторы расположены в `hybrid/`, где поверх объединённых TF-IDF и BERT-векторов обучаются классические модели и MLP-классификатор.

8.6 Воспроизводимость экспериментов

Для исследовательской работы важна воспроизводимость экспериментов, поэтому в реализации фиксируются основные параметры подготовки данных, обучения и оценки моделей. Train/test-разбиение создаётся с фиксированным `random_state=42`, что позволяет повторно получать одинаковое разделение данных. Во многих ветках также используется фиксированный `seed` для генераторов случайных чисел.

В пайплайне сохраняются промежуточные результаты: очищенный датафрейм, `train/test`-файлы, отдельные аугментированные наборы, объединённый аугментированный набор, суммаризованные варианты и построенные признаки. Это позволяет не пересчитывать каждый раз все этапы с нуля и запускать отдельные эксперименты независимо.

Конфигурационные параметры вынесены в отдельные файлы или явно заданы в скриптах. Например, для BERT-классификатора отдельно задаются параметры модели, данных и обучения: максимальная длина чанка, `stride`, число чанков, `learning rate`, `dropout`, `label smoothing`, число эпох и параметры ранней остановки. В классических моделях зафиксированы параметры TF-IDF-векторизации и классификаторов.

Дополнительно в репозитории хранится сводка экспериментальных результатов. Это позволяет сопоставлять разные ветки моделирования и отслеживать, на каких данных и с какими настройками были получены результаты. Такая организация делает пайплайн не только исследовательским, но и воспроизводимым: каждый этап можно повторить или модифицировать без нарушения общей структуры проекта.

8.7 Ограничения реализации и требования к вычислительным ресурсам

Разработанный пайплайн включает как лёгкие классические модели, так и более ресурсоёмкие BERT-подходы. Классическая ветка TF-IDF + LinearSVC или LogisticRegression может обучаться на CPU и не требует значительного объёма видеопамати. Это делает её удобной для быстрых экспериментов, проверки данных и получения базовой линии качества.

BERT-подходы предъявляют более высокие требования к вычислительным ресурсам. Особенно это относится к `ai-forever/ruRoberta-large`, которая используется в задачах классификации, построения эмбедингов и обработки длинных документов. При

работе с чанкингом один документ превращается в несколько фрагментов, поэтому фактическая нагрузка на память и время обучения возрастает.

Для уменьшения требований к памяти в реализации используются малый batch size и накопление градиентов. В основном BERT-классификаторе применяется batch_size=1 и grad_accum_steps=8, что позволяет обучать модель при ограниченной видеопамяти. Также используются gradient checkpointing, ограничение числа чанков на документ и ранняя остановка обучения.

Отдельное ограничение связано со временем генерации аугментированных данных. Перефразирование и обратный перевод выполняются на уровне фрагментов, причём для каждого фрагмента может генерироваться несколько кандидатов. Это повышает качество отбора, но увеличивает время подготовки аугментированного корпуса. Поэтому аугментация реализована как отдельный этап, результаты которого сохраняются в файлы и затем переиспользуются.

Таким образом, программная реализация поддерживает несколько уровней вычислительной сложности. Быстрые классические модели позволяют оперативно проверять гипотезы, а более тяжёлые BERT- и hybrid-подходы используются для исследования контекстных и композитных представлений текста. Такая структура позволяет работать с задачей в условиях ограниченных ресурсов и при этом сохранять возможность расширения пайплайна. Весь код доступен по адресу <https://github.com/sevaTechoPark/mifi-vkr/tree/main>

9 Экспериментальное исследование и сравнение результатов

9.1 Экспериментальный протокол оценки качества

Экспериментальное исследование проводилось на фиксированном train/test-разбиении, сформированном после очистки и нормализации исходного датасета. Обучающая выборка использовалась для построения признаков, обучения моделей, аугментации и формирования дополнительных вариантов данных. Тестовая выборка оставалась неизменной и применялась только для итоговой оценки качества.

В рамках экспериментов сравнивались несколько групп подходов. Первая группа включала классический baseline на основе TF-IDF-признаков и линейного классификатора. Вторая группа была основана на BERT-классификации с использованием ruRoberta-large. Отдельно рассматривались подходы на основе косинусного сходства в пространстве эмбедингов и гибридные модели, объединяющие TF-IDF- и BERT-представления.

Для проверки влияния состава обучающих данных использовались четыре варианта train-набора: исходный текст, аугментированный текст, суммаризованный текст и комбинированный вариант, где исходный документ объединялся с его кратким содержанием. Такой протокол позволил сравнить не только модели, но и способы представления исходного письма.

Все результаты далее приводятся в едином формате: модель обучается на выбранном варианте train-набора и оценивается на одной и той же тестовой выборке. Это делает сравнение между подходами корректным и позволяет анализировать влияние отдельных компонентов пайплайна: очистки, аугментации, суммаризации, выбора модели и способа построения признаков.

9.2 Используемые метрики качества классификации

Для оценки качества использовались метрики, подходящие для многоклассовой классификации при несбалансированном распределении классов. Основное внимание уделялось balanced accuracy и macro F1, поскольку они учитывают качество работы модели по всем классам и не позволяют крупным категориям полностью доминировать в итоговой оценке.

Balanced accuracy отражает среднюю полноту по классам. Эта метрика особенно важна для данной задачи, так как в датасете есть как крупные категории, так и редкие классы. Если модель хорошо распознаёт только наиболее частые классы, balanced accuracy будет ниже, чем обычная accuracy, и лучше покажет реальную устойчивость классификатора.

Macro F1 также рассчитывается с равным вкладом каждого класса. Она учитывает одновременно точность и полноту, поэтому полезна при сравнении моделей на несбалансированных данных. В данной работе macro F1 используется как одна из основных метрик, поскольку важно оценить не только общий процент правильных ответов, но и способность модели работать со всем множеством классов.

9.3 Сравнение качества на исходной и аугментированной обучающей выборке

Первым направлением анализа стало сравнение качества моделей на исходной и аугментированной обучающей выборке. Аугментация применялась для увеличения числа примеров в редких классах и формирования более сбалансированного train-набора. Поэтому её влияние оценивалось отдельно для классического baseline и BERT-классификатора.

Для TF-IDF baseline результаты на исходном и аугментированном наборе оказались заметно различными: на оригинальных данных и balanced accuracy, и macro F1 выше. Это объясняется тем, что текущий чистый train — уже отфильтрованный, высококачественный набор, где каждый документ несёт реальную смысловую нагрузку и формирует плотное, чистое TF-IDF-пространство. После аугментации добавляются синтетические примеры, которые, хотя и увеличивают лексическое разнообразие, «размазывают» границы между классами в пространстве признаков. Такие промежуточные тексты усложняют задачу линейному классификатору и приводят к падению качества на тесте.

Для BERT-классификаторов картина обратная: во всех рассмотренных вариантах (ruBERT и ruRoberta, CLS и mean pooling, включая chunkmean) обучение на аугментированном train-наборе приводит к росту и balanced accuracy, и macro F1. Это согласуется с гипотезой, что нейросетевые модели чувствительны к разнообразию формулировок и способны извлекать полезный сигнал из синтетических примеров, улучшая обобщающую способность на тестовой выборке.

Таблица 9.1 — Влияние аугментации на TF-IDF baseline и BERT-классификатор

Подход	Обучающие данные	Balanced accuracy	Macro F1
TF-IDF baseline	train	0.534	0.543
TF-IDF baseline	train augmented	0.489	0.486
rubert-base-cased CLS	train	0.359	0.347
rubert-base-cased CLS	train augmented	0.433	0.431
rubert-base-cased MeanPooling	train	0.406	0.408
rubert-base-cased MeanPooling	train augmented	0.453	0.448
ruRoberta-large CLS	train	0.388	0.377
ruRoberta-large CLS	train augmented	0.507	0.488
ruRoberta-large MeanPooling	train	0.396	0.394
ruRoberta-large MeanPooling	train augmented	0.479	0.469
ruRoberta-large chunkmean	train	0.421	0.393
ruRoberta-large chunkmean	train augmented	0.500	0.504

Таким образом, аугментация не является универсальным способом улучшения любого классификатора. Её эффект зависит от модели и типа признакового пространства. В данной работе она оказалась особенно полезной для BERT-подхода, поскольку позволила расширить множество формулировок, доступных модели при обучении.

9.4 Сравнение исходных, суммаризованных и комбинированных текстовых представлений

Отдельно исследовалось влияние суммаризации. Для этого сравнивались три варианта представления документа: исходный текст, только суммаризованный и комбинированное представление original+summary. Такой эксперимент был важен из-за большой длины деловых писем и ограничений BERT-моделей по числу входных токенов.

Таблица 9.2 — Влияние суммаризации на TF-IDF baseline

Обучающие данные	Balanced accuracy	Macro F1
train_summarized	0.366	0.378
train original plus summary	0.482	0.497
train augmented summarized	0.384	0.395
train augmented original plus summary	0.467	0.475

Для TF-IDF baseline все варианты, в которых исходный текст заменяется или дополняется суммаризацией, дают более низкие метрики по сравнению с обучением на чистом исходном train-наборе. При этом внутри самих суммаризованных конфигураций комбинированное представление original+summary заметно лучше, чем обучение только на summary: и для неаугментированных данных (0.482 против 0.366 по balanced accuracy), и для аугментированных (0.467 против 0.384). Это говорит о том, что baseline нуждается в полноте исходного текста и теряет важные признаки при переходе к короткому резюме, но сочетание оригинала и краткого содержания частично компенсирует эту потерю.

Таблица 9.3 — Суммаризированный датасет на BERT-классификатор

Подход	Обучающие данные	Balanced accuracy	Macro F1
rubert-base-cased CLS	train_summarized	0.193	0.183
rubert-base-cased MeanPooling	train_summarized	0.242	0.240
ruRoberta-large CLS	train_summarized	0.345	0.325
ruRoberta-large MeanPooling	train_summarized	0.333	0.326
rubert-base-cased CLS	train_augmented_summarized	0.326	0.291
rubert-base-cased MeanPooling	train_augmented_summarized	0.270	0.271
ruRoberta-large CLS	train_augmented_summarized	0.379	0.383
ruRoberta-large MeanPooling	train_augmented_summarized	0.411	0.4125

Для BERT-классификаторов обучение только на суммаризованных текстах также приводит к ухудшению качества по сравнению с исходными письмами. На чистом train_summarized и ruBERT, и ruRoberta показывают заметно более низкие значения balanced accuracy и macro F1, чем при обучении на полном тексте. Подключение аугментации к суммаризованным данным улучшает метрики, особенно для ruRoberta, но даже в этом случае результаты остаются ниже, чем у моделей, обученных на полном тексте с аугментацией. В совокупности это показывает, что для текущей задачи суммаризация в виде самостоятельной замены исходного письма не даёт выигрыша ни для TF-IDF baseline, ни для BERT-классификаторов, а наилучшее качество достигается при работе с максимально богатым по содержанию представлением письма.

Таблица 9.4 — Комбинированный датасет на BERT-классификатор

Подход	Обучающие данные	Balanced accuracy	Macro F1
rubert-base-cased CLS	train_original_plus_summary	0.363	0.358
rubert-base-cased MeanPooling	train_original_plus_summary	0.3755	0.386
ruRoberta-large CLS	train_original_plus_summary	0.383	0.381

ruRoberta-large MeanPooling	train_original_plus_summary	0.444	0.435
rubert-base-cased CLS	train_augmented_original_plus_summary	0.420	0.423
rubert-base-cased MeanPooling	train_augmented_original_plus_summary	0.461	0.452
ruRoberta-large CLS	train_augmented_original_plus_summary	0.478	0.459
ruRoberta-large MeanPooling	train_augmented_original_plus_summary	0.511	0.498

Для BERT-классификаторов комбинированное представление original+summary оказывается промежуточным по качеству между обучением только на полном тексте и обучением только на суммаризованных письмах. Без аугментации метрики для train_original_plus_summary заметно ниже, чем для исходного train из предыдущих экспериментов, но выше, чем для train_summarized. Подключение аугментации к комбинированному датасету стабильно улучшает результаты для всех вариантов BERT: на train_augmented_original_plus_summary и balanced accuracy, и macro F1 возрастают, а лучшая конфигурация (ruRoberta-large с mean pooling) достигает 0.511 по balanced accuracy и 0.498 по macro F1. Это уже сопоставимо с лучшими значениями для моделей, обученных на полном тексте с аугментацией, что показывает, что связка «исходное письмо + краткое резюме» в сочетании с аугментацией даёт BERT-классификатору дополнительный полезный сигнал и может использоваться как конкурентоспособный вариант представления документа.

9.5 Сравнение классических, нейросетевых и композитных представлений

Косинусные подходы и hybrid-модели были включены для проверки эмбедингового и композитного представления текстов. Они позволяют исследовать, насколько документы одного класса сближаются в векторном пространстве и можно ли объединить сильные стороны TF-IDF и BERT. Они важны для анализа свойств признакового пространства, но простое объединение признаков не обязательно улучшает классификацию.

Таблица 9.5 — Сравнение методов косинусного расстояния на эмбединговом представлении текста

Эмбединги	Обучающие данные	Метод	Balanced accuracy	Macro F1
Предобученные	train	Центроид	0.125	0.095
Fine-tuned	train	Центроид	0.564	0.499
Предобученные	train	Ближайший сосед	0.176	0.177
Fine-tuned	train	Ближайший сосед	0.476	0.475
Предобученные	train_augmented	Центроид	0.103	0.103
Fine-tuned	train_augmented	Центроид	0.476	0.470
Предобученные	train_augmented	Ближайший сосед	0.197	0.186

Fine-tuned	train_augmented	Ближайший сосед	0.476	0.487
------------	-----------------	-----------------	-------	-------

Таблица 9.6 — mlr на композитных представлений

Сделать

Таблица 9.7 — классические методы на композитных представлений

Сделать

9.6 Сводная таблица результатов лучших реализованных подходов

Основная таблица отражает результаты блока «Финал» из экспериментальной сводки.

Таблица 9.8 — Сводная таблица результатов лучших подходов

Сюда добавить лучшие датасеты и лучшие методы(всё что 0.5+)

Из таблицы видно, что наиболее устойчивыми группами оказались TF-IDF baseline и BERT-classification. Косинусные и гибридные подходы были полезны как исследовательские ветки для проверки структуры эмбедингового пространства и композитных признаков, однако в итоговом сравнении они уступили основным методам.

Отдельно следует отметить результат улучшенной очистки данных. Этот результат не смешивается с основной таблицей, поскольку относится к другой версии предобработки, но показывает значимость этапа очистки данных для всей задачи.

Таблица 9.9 — Результаты baseline для разных версий очистки данных

Сюда вставить три версии(изначальную, вторую и последнюю)

9.7 Выводы по результатам экспериментального исследования

В ходе экспериментального исследования были реализованы и сопоставлены несколько подходов к классификации длинных русскоязычных деловых писем: классический TF-IDF baseline, BERT-classification, методы косинусного сравнения эмбедингов и hybrid-представления. Сравнение проводилось на нескольких вариантах обучающих данных: original, augmented, summarized и original+summary.

Результаты показали, что суммаризация как полная замена исходного текста не является оптимальной для данной задачи. При использовании только summary качество снижалось как у TF-IDF baseline, так и у BERT-classification. Это связано с тем, что в деловых письмах важными могут быть короткие предметные детали, которые не всегда сохраняются в кратком содержании.

Результаты показали, что суммаризация как полная замена исходного текста не является оптимальной для данной задачи. При использовании только summary качество снижалось как у TF-IDF baseline, так и у BERT-classification. Это связано с тем, что в деловых письмах важными могут быть короткие предметные детали, которые не всегда сохраняются в кратком содержании.

Аугментация показала разный эффект для разных моделей. Для TF-IDF baseline она не дала заметного прироста, тогда как для BERT-classification улучшила обе основные метрики. Это подтверждает, что методы расширения данных должны оцениваться в связке с конкретным типом модели и признакового пространства.

Отдельно важным результатом является влияние очистки данных. После доработки предобработки TF-IDF baseline на исходном train-наборе достиг balanced accuracy 0.534 и

масго F1 0.543. Этот результат показывает, что для рассматриваемой задачи качество подготовки текстов оказывает существенное влияние на итоговую классификацию.

Таким образом, основным итогом экспериментальной части является построение и сравнение нескольких взаимодополняющих веток решения. Для данной задачи эффективными оказались как классические лексические признаки, так и контекстные модели при правильном выборе представления данных. Полученные результаты подтверждают необходимость рассматривать классификацию длинных деловых писем как полный пайплайн, включающий очистку, аугментацию, суммаризацию, построение признаков и выбор модели.

ЗАКЛЮЧЕНИЕ

В выпускной квалификационной работе была рассмотрена задача многоклассовой классификации длинных русскоязычных деловых писем. В качестве исходных данных использовался корпоративный датасет, содержащий 36 классов и характеризующийся выраженным дисбалансом, различной длиной документов и наличием предметно-специфичной деловой лексики. В ходе работы был реализован полный исследовательский пайплайн: от очистки и анализа данных до аугментации, построения различных текстовых представлений, обучения моделей и итогового сравнения результатов.

На этапе предобработки были изучены структура датасета, распределение классов, длина текстов, наличие дубликатов, пересечений и аномальных объектов. Были реализованы правила очистки и нормализации, направленные на удаление технического шума при сохранении предметно значимых элементов: названий проектов, подразделений, аббревиатур и устойчивых деловых формулировок. После очистки число классов сохранилось, а тексты были приведены к более однородному виду, пригодному для дальнейшего построения признаков и обучения моделей.

Для уменьшения влияния дисбаланса была реализована аугментация обучающей выборки. В работе применялись классические методы обработки текста, перефразирование и обратный перевод. Для длинных документов была разработана схема фрагментарной генерации: текст разбивался на предложения и фрагменты, для них создавались кандидаты аугментации, после чего фрагменты склеивались обратно в полный документ. Для контроля качества использовалась фильтрация по косинусному сходству, что позволило отбирать варианты, близкие по смыслу к исходному тексту, но не являющиеся его точной копией.

В качестве классического направления был реализован пайплайн на основе TF-IDF-признаков и линейных классификаторов. Были проверены word n-граммы, char n-граммы и их объединение в едином признаковом пространстве. Поверх этих признаков использовались LinearSVC, LogisticRegression и дополнительные классические модели. Такой подход показал себя как сильная и вычислительно эффективная базовая линия, особенно после улучшения предобработки данных.

Нейросетевое направление было представлено BERT-подобными моделями. В работе рассматривались ruBERT и ruRoberta-large, варианты классификации через стандартную голову, mean pooling, Sentence-BERT-представления, а также отдельный классификатор ruRoberta-large для длинных документов с разбиением текста на перекрывающиеся чанки. Для стабилизации обучения использовались веса классов, label smoothing, dropout, накопление градиентов и ранняя остановка.

Отдельно были исследованы эмбединговые и композитные представления текста. Для документов строились BERT-эмбединги на основе предобученной и дообученной модели, после чего применялись классические классификаторы, классификация по центроидам и метод ближайшего соседа. Также была реализована гибридная ветка, в которой TF-IDF-признаки объединялись с BERT-векторами, а поверх композитного пространства обучались линейные модели и MLP-классификатор.

В экспериментальной части были сопоставлены исходные, аугментированные, суммаризованные и комбинированные представления данных. Было показано, что использование только summary как замены исходного текста снижает качество классификации, поскольку краткое содержание может терять важные предметные детали. При этом комбинированный вариант original+summary оказался полезным для BERT-классификатора, так как позволил использовать одновременно полный текст и его сжатое смысловое представление.

По итогам основной сводки TF-IDF baseline на исходных данных показал balanced accuracy 0,453 и macro F1 0,457, а BERT-classification на варианте original+summary достиг balanced accuracy 0,456 и macro F1 0,469. Дополнительно было установлено, что доработка

очистки данных существенно влияет на качество классического подхода: обновлённый TF-IDF baseline на очищенном train-наборе достиг balanced accuracy 0,534 и macro F1 0,543. Это подтверждает, что для задач классификации деловой переписки качество подготовки данных является одним из ключевых факторов.

Главным результатом работы является разработка модульного программного прототипа для классификации длинных деловых писем. Прототип включает этапы предобработки, аугментации, суммаризации, построения TF-IDF, BERT и композитных представлений, обучения моделей и сравнения результатов. Модульная структура позволяет повторно запускать отдельные эксперименты, добавлять новые методы и использовать разные варианты обучающих данных без полной переработки проекта.

Практическая значимость работы заключается в том, что предложенный пайплайн может быть адаптирован для задач автоматической маршрутизации корпоративных писем, категоризации внутренних документов и поддержки процессов электронного документооборота. Дальнейшее развитие решения может быть связано с расширением обучающей выборки, более детальным анализом ошибок по классам, настройкой стратегий аугментации, применением специализированных моделей для длинных документов и улучшением композитных признаков представлений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Статьи и материалы конференций:

1. Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A.N., Kaiser L., Polosukhin I. Attention Is All You Need // Advances in Neural Information Processing Systems. — 2017. — URL: <https://www.semanticscholar.org/paper/204e3073870fae3d05bcb2f6a8e263d9b72e776>.
2. Devlin J., Chang M.-W., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding // NAACL-HLT. — 2019. — P. 4171–4186. — URL: <https://aclanthology.org/N19-1423>.
3. Liu Y., Ott M., Goyal N., Du J., Joshi M., Chen D., Levy O., Lewis M., Zettlemoyer L., Stoyanov V. RoBERTa: A Robustly Optimized BERT Pretraining Approach. — 2019. — URL: <https://www.semanticscholar.org/paper/077f8329a7b6fa3b7c877a57b81eb6c18b5f87de>.
4. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks // EMNLP-IJCNLP. — 2019. — P. 3982–3992. — URL: <https://aclanthology.org/D19-1410>.
5. Kuratov Y., Arkhipov M. Adaptation of Deep Bidirectional Multilingual Transformers for Russian Language. — 2019. — URL: <https://arxiv.org/abs/1905.07213>.
6. Malykh V., et al. A Family of Pretrained Transformer Language Models for Russian. — 2023. — URL: <https://arxiv.org/abs/2309.10931>.
7. Beltagy I., Peters M.E., Cohan A. Longformer: The Long-Document Transformer. — 2020. — URL: <https://www.semanticscholar.org/paper/925ad2897d1b5decbea320d07e99afa9110e09b2>.
8. Wei J., Zou K. EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks // EMNLP-IJCNLP. — 2019. — P. 6382–6388. — URL: <https://aclanthology.org/D19-1670>.
9. Karimi A., Rossi L., Prati A. AEDA: An Easier Data Augmentation Technique for Text Classification // Findings of EMNLP. — 2021. — P. 2748–2754. — URL: <https://aclanthology.org/2021.findings-emnlp.234>.
10. Ciolino M., Noever D., Kalin J. Back Translation Survey for Improving Text Augmentation. — 2021. — URL: <https://www.semanticscholar.org/paper/34e6f0b17a2d413a68195692d3e35490a4e8a455>.
11. Jiang Z., Gao B., He Y., Han Y., Doyle P., Zhu Q. Text Classification Using Novel Term Weighting Scheme-Based Improved TF-IDF for Internet Media Reports // Mathematical Problems in Engineering. — 2021. — URL: <https://www.hindawi.com/journals/mpe/2021/6619088/>.
12. Watanangura P., Vanichrudee S., Minter O., Sringamdee T., Thanngam N., Siriborvornratanakul T. A Comparative Survey of Text Summarization Techniques // SN Computer Science. — 2023. — URL: <https://link.springer.com/10.1007/s42979-023-02343-6>.
13. Myla S.D., Saini R., Kapoor N. Auto Text Summarization in Natural Language Processing: Review // IDCIoT. — 2024. — URL: <https://ieeexplore.ieee.org/document/10467376/>.
14. Taskiran S.F., Turkoglu B., Kaya E., Aşuroğlu T. A comprehensive evaluation of oversampling techniques for enhancing text classification performance // Scientific Reports. — 2025. — URL: <https://www.nature.com/articles/s41598-025-05791-7>.
15. Hinojosa Lee M.C., Braet J., Springael J. Performance Metrics for Multilabel Emotion Classification: Comparing Micro, Macro, and Weighted F1-Scores // Applied Sciences. — 2024. — URL: <https://www.mdpi.com/2076-3417/14/21/9863>.

Книги и учебные издания:

1. Грон А. Машинное обучение. Наука и искусство построения алгоритмов, которые извлекают знания из данных. — СПб.: Питер, 2020. — 400 с.
2. Жуков Л.А. Машинное обучение и анализ данных. — М.: МФТИ, 2019. — 520 с.
3. Jurafsky D., Martin J.H. Speech and Language Processing. 3rd ed. — Draft. — 2023. — URL: <https://web.stanford.edu/~jurafsky/slp3/>.
4. Goldberg Y. Neural Network Methods for Natural Language Processing. — San Rafael: Morgan & Claypool, 2017. — 310 p.

Электронные ресурсы и документация:

1. DeepPavlov/rubert-base-cased: model card [Электронный ресурс]. — URL: <https://huggingface.co/DeepPavlov/rubert-base-cased> (дата обращения: 17.05.2026).
2. DeepPavlov/rubert-base-cased-sentence: model card [Электронный ресурс]. — URL: <https://huggingface.co/DeepPavlov/rubert-base-cased-sentence> (дата обращения: 17.05.2026).
3. ai-forever/ruRoberta-large: model card [Электронный ресурс]. — URL: <https://huggingface.co/ai-forever/ruRoberta-large> (дата обращения: 17.05.2026).
4. BERT in DeepPavlov documentation [Электронный ресурс]. — URL: <https://docs.deeppavlov.ai/en/master/features/models/bert.html> (дата обращения: 17.05.2026).
5. scikit-learn: TfidfVectorizer documentation [Электронный ресурс]. — URL: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html (дата обращения: 17.05.2026).
6. scikit-learn: LinearSVC documentation [Электронный ресурс]. — URL: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html> (дата обращения: 17.05.2026).
7. scikit-learn: LogisticRegression documentation [Электронный ресурс]. — URL: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html (дата обращения: 17.05.2026).

Нормативные документы:

1. ГОСТ 7.32—2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. — М.: Стандартинформ, 2018. — 24 с.
2. ГОСТ Р 7.0.5—2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления. — М.: Стандартинформ, 2008. — 18 с.
3. ГОСТ 7.1—2003. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — М.: Стандартинформ, 2003. — 47 с.